Politechnika Warszawska

WYDZIAŁ ELEKTRYCZNY

Instytut Sterowania i Elektroniki Przemysłowej

Praca dyplomowa inżynierska

na kierunku Informatyka Stosowana

w specjalności Inżynieria danych i multimedia

System do przetwarzania i archiwizacji strumieniowych danych finansowych w środowisku lokalnym

Maciej Karabin
numer albumu 325483

promotor
mgr inż. Krzysztof Marek

WARSZAWA 2026

System do przetwarzania i archiwizacji strumieniowych danych finansowych w środowisku lokalnym

Streszczenie

Praca przedstawia projekt oraz implementację systemu do przetwarzania i archiwizacji strumieniowych danych finansowych z rynku kryptowalut w środowisku lokalnym. Głównym celem jest zapewnienie wiarygodnego i kompletnego łańcucha przetwarzania danych, od odbioru, przez walidację i wzbogacanie, aż po udostępnianie i archiwizację w trybie ciągłym.

W ramach pracy zrealizowano architekturę opartą na mikrousługach, która integruje technologie takie jak Go, Docker, NATS JetStream, MongoDB oraz MinIO. System pobiera dane w czasie rzeczywistym z giełdy Binance za pomocą dedykowanego serwera strumieniowego. Następnie komunikaty są dystrybuowane przez brokera NATS JetStream do modułów ETL, które odpowiadają za ich transformację, oczyszczanie oraz wzbogacanie o metadane analityczne. Kluczowym elementem systemu jest mechanizm pamięci warstwowej (*tiered storage*), który automatycznie przenosi dane historyczne z operacyjnej bazy danych MongoDB do archiwalnego magazynu obiektowego MinIO, optymalizując koszty i wydajność. Dostęp do surowych danych zapewnia interfejs GraphQL, a zintegrowany z platformą Telegram bot umożliwia wysyłanie powiadomień o zdarzeniach rynkowych.

Struktura pracy obejmuje analizę istniejących rozwiązań i wymagań systemowych w rozdziale drugim. Rozdział trzeci szczegółowo opisuje zaprojektowaną architekturę systemu i przepływ danych między komponentami. Implementacja poszczególnych komponentów oraz wykorzystane technologie zostały przedstawione w rozdziale czwartym. Rozdział piąty wskazuje możliwe kierunki dalszego rozwoju systemu. Rozdział szósty dokumentuje przeprowadzone testy manualne weryfikujące poprawność działania systemu. Pracę kończy podsumowanie zawierające wnioski z realizacji projektu.

Słowa kluczowe: przetwarzanie strumieniowe, kryptowaluty, object storage, ETL, pamięć warstwowa, Go, NATS JetStream, EDA

A system for processing and archiving real-time financial data in an on-premises environment

Abstract

The thesis presents the design and implementation of a system for processing and archiving streaming financial data from the cryptocurrency market in an on-premises environment. The main objective is to ensure a reliable and complete data processing chain, from ingestion, through validation and enrichment, to continuous delivery and archiving.

The project implements a microservices-based architecture that integrates technologies such as Go, Docker, NATS JetStream, MongoDB, and MinIO. The system fetches real-time data from the Binance exchange using a dedicated streaming server. Subsequently, the NATS JetStream broker distributes messages to ETL modules, which are responsible for their transformation, cleansing, and enrichment with analytical metadata. A key element of the system is the tiered storage mechanism, which automatically moves historical data from the operational MongoDB database to the archival MinIO object storage, optimizing costs and performance. Access to raw data is provided by a GraphQL interface, and a Telegram bot integrated with the platform enables sending notifications about market events.

The structure of the thesis includes an analysis of existing solutions and system requirements in the second chapter. The third chapter describes the designed system architecture and data flow between components in detail. The implementation of individual components and the technologies used are presented in the fourth chapter. The fifth chapter indicates possible directions for further development of the system. The sixth chapter documents the manual tests conducted to verify the correct operation of the system. The thesis concludes with a summary that includes conclusions from the project.

Keywords: stream processing, cryptocurrency, object storage, ETL, tiered storage, Go, NATS JetStream, Event Driven Architecture

Spis treści

1 Wstęp **1**

1.1 Wprowadzenie . . . 1

1.2 Cel pracy . . . 2

1.3 Plan pracy . . . 2

2 Przetwarzanie danych strumieniowych **5**

2.1 Charakterystyka danych strumieniowych . . . 5

2.2 Modele przetwarzania danych . . . 5

2.2.1 Przetwarzanie wsadowe . . . 6

2.2.2 Przetwarzanie strumieniowe . . . 6

2.2.3 Architektura Lambda i Kappa . . . 6

2.3 Semantyka dostarczania komunikatów . . . 7

2.3.1 At-most-once (co najwyżej raz) . . . 7

2.3.2 At-least-once (co najmniej raz) . . . 7

2.3.3 Exactly-once (dokładnie raz) . . . 7

2.4 Okna czasowe w przetwarzaniu strumieniowym . . . 8

2.4.1 Okna stałe (fixed windows) . . . 8

2.4.2 Okna przesuwne (sliding windows) . . . 8

2.4.3 Okna sesyjne (session windows) . . . 8

2.5 Problemy i wyzwania . . . 9

2.5.1 Zarządzanie czasem zdarzeń . . . 9

2.5.2 Zarządzanie stanem . . . 9

2.5.3 Backpressure i kontrola przepływu . . . 9

2.5.4 Skalowalność i partycjonowanie . . . 10

2.6 Architektura sterowana zdarzeniami . . . 10

2.7 Podsumowanie . . . 11

3 Analiza wymagań **13**

3.1 Założenia projektu . . . 13

3.2 Wymagania funkcjonalne . . . 14

3.2.1 Pobieranie danych . . . 14
3.2.2 Przechowywanie danych . . . 15
3.2.3 Przetwarzanie danych . . . 15
3.2.4 Udostępnianie danych i powiadamianie . . . 15
3.3 Wymagania niefunkcjonalne . . . 16
3.3.1 Wydajność i skalowalność . . . 16
3.3.2 Niezawodność i odporność na awarie . . . 17
3.4 Istniejące rozwiązania . . . 17
3.4.1 Binance-Data-Collector-and-Visualization . . . 17
3.4.2 binance_ticker_rust . . . 18
3.4.3 Binance Datatool (BHDS) . . . 18
3.4.4 binance-websocket . . . 18
3.4.5 Wnioski z analizy porównawczej . . . 19

4 Projekt systemu **21**
4.1 Serwer strumieniowy (Data Fetcher) . . . 21
4.2 Broker wiadomości (NATS JetStream) . . . 22
4.3 Warstwa gorąca (MongoDB) . . . 22
4.4 Moduły ETL (Extract, Transform, Load) . . . 22
4.5 Warstwa zimna (MinIO Object Storage) . . . 23
4.6 Interfejs GraphQL . . . 23
4.7 Serwis alertów (Telegram Bot) . . . 23
4.8 Analityka strumieniowa i historyczna . . . 24

5 Implementacja **25**
5.1 Wykorzystane technologie . . . 25
5.1.1 Język programowania Go . . . 25
5.1.2 Docker i Docker Compose . . . 25
5.1.3 NATS JetStream . . . 26
5.1.4 MongoDB . . . 26
5.1.5 MinIO . . . 28
5.1.6 GraphQL i gqlgen . . . 29
5.1.7 Modele danych w języku Go . . . 30
5.1.8 Format Parquet . . . 30
5.1.9 Viper . . . 31
5.1.10 Jupyter Notebook i Python . . . 31
5.2 Problemy i wyzwania . . . 31
5.2.1 Złożoność architektury mikrousługowej . . . 31
5.2.2 Debugowanie i śledzenie rozproszone . . . 31

5.2.3 Przetwarzanie sterowane zdarzeniami . . . 32
5.2.4 Zarządzanie wieloma kontenerami . . . 32
5.2.5 Jednoczesny dostęp do danych historycznych i bieżących . . . 32
5.2.6 Projektowanie elastycznego API . . . 32
5.2.7 Optymalizacja procesu archiwizacji . . . 33
5.3 Możliwości dalszego rozwoju . . . 33
5.3.1 Centralizacja logów . . . 33
5.3.2 Metryki i dashboardy (Prometheus/Grafana) . . . 33
5.3.3 Śledzenie rozproszone (distributed tracing) . . . 33
5.3.4 Alerty operacyjne i powiadomienia o stanie systemu . . . 34
5.3.5 Rozszerzenie API: dostęp do danych z MinIO oraz federacja GraphQL . . . 34
5.3.6 Bardziej zaawansowany moduł analityczny . . . 34
5.3.7 Modele predykcyjne i pipeline uczenia maszynowego . . . 34
5.3.8 Dalsza optymalizacja i skalowanie do wszystkich symboli . . . 34

6 Weryfikacja rozwiązania 37
6.1 Test T1: Odbiór danych z Binance WebSocket . . . 38
6.2 Test T2: Publikowanie zdarzeń do NATS JetStream . . . 39
6.3 Test T3: Subskrybowanie zdarzeń przez moduł MongoDB . . . 40
6.4 Test T4: Subskrybowanie zdarzeń przez moduł Telegram Bot . . . 41
6.5 Test T5: Wysyłanie powiadomienia Telegram – filtr spełniony . . . 42
6.6 Test T6: Brak powiadomienia Telegram – filtr niespełniony . . . 43
6.7 Test T7: Zapytanie GraphQL z filtrami zwracającymi wyniki . . . 44
6.8 Test T8: Zapytanie GraphQL z filtrami niezwracającymi wyników . . . 45
6.9 Test T9: Proces ETL i mechanizm tiered storage . . . 46

7 Podsumowanie 49

Bibliografia 50

Załączniki 55

Rozdział 1

Wstęp

1.1 Wprowadzenie

Dynamiczny rozwój rynku kryptowalut oraz jego wysoka zmienność powodują, że organizacje finansowe potrzebują wiarygodnych strumieni danych o niskim opóźnieniu. Rynek ten generuje dzienny wolumen transakcyjny rzędu dziesiątek miliardów dolarów [1]. Jednocześnie specyfika giełd cyfrowych niesie ze sobą ryzyka związane z jakością danych. Zjawiska takie jak *wash trading*, czyli sztuczne generowanie obrotu, mogą prowadzić do zafałszowania informacji rynkowych, przez co publicznie dostępne, niesprawdzone dane mogą okazać się niewiarygodne [2]. W konsekwencji podmioty analizujące rynek, w szczególności w kontekście wykrywania anomalii czy okazji arbitrażowych, nie mogą opierać się wyłącznie na zewnętrznych agregatorach, lecz powinny budować własne potoki przetwarzania danych (*data pipelines*) [3].

Ciągły dostęp do danych historycznych i strumieniowych jest fundamentem algorytmicznych strategii inwestycyjnych. W środowiskach o ograniczonym zaufaniu do sieci zewnętrznej istotna staje się także możliwość autonomicznego działania systemu (*disconnected operations*), co wpisuje się w założenia przetwarzania brzegowego (*edge computing*) [4].

Projektowana architektura obejmuje serwer strumieniowy nasłuchujący kanałów WebSocket giełdy Binance, usługę wymiany komunikatów JetStream, moduły *ETL* odpowiedzialne za transformację i wzbogacanie danych oraz warstwę analityczną udostępniającą zagregowane metryki. Dostęp do danych zapewnia interfejs GraphQL, natomiast bot platformy Telegram odpowiada za dystrybucję powiadomień o zdarzeniach rynkowych w czasie rzeczywistym.

Kluczowym elementem infrastruktury jest mechanizm *tiered storage* (pamięć warstwowa). Dane w systemie trafiają najpierw do warstwy gorącej (MongoDB), a następnie – po upływie określonego czasu retencji – przenoszone są do warstwy zimnej opartej na MinIO. Pozwala to odciążyć bazę operacyjną, obniżyć koszty przechowywania danych historycznych oraz zwiększyć niezawodność przetwarzania [5]. W warstwie obiektowej dane mogą być dodatkowo kompresowane i wersjonowane.

1.2 Cel pracy

Celem pracy jest zaprojektowanie i implementacja kompletnego łańcucha przetwarzania danych rynkowych obejmującego odbiór, walidację, wzbogacanie, udostępnianie oraz archiwizację informacji w trybie ciągłym. W ramach pracy został opracowany niezawodny serwer strumieniowy utrzymujący połączenia z WebSocket Binance. Zaprojektowano kanał dystrybucyjny oparty na NATS JetStream, zapewniający buforowanie zdarzeń oraz odporność na awarie usług zależnych. Zaimplementowano procesy *ETL* dzielące dane na zbiory surowe i oczyszczone oraz wzbogacające je o metadane analityczne. Skonfigurowano i zautomatyzowano przenoszenie danych w modelu *tiered storage* (MongoDB → MinIO), zapewniając optymalizację zasobów warstwy operacyjnej przy równoczesnej dostępności danych historycznych. Zintegrowano system powiadomień z Telegramem, umożliwiając przesyłanie alertów rynkowych w czasie rzeczywistym.

Realizacja założeń obejmuje przygotowanie dokumentacji architektury, implementację usług w języku *Go* oraz konfigurację infrastruktury w środowisku Docker.

1.3 Plan pracy

Niniejsza praca składa się z siedmiu rozdziałów, z czego pierwszy stanowi wprowadzenie do problematyki, kolejne cztery opisują realizację systemu, zaś dwa ostatnie podsumowują osiągnięte rezultaty oraz wskazują kierunki dalszego rozwoju.

W rozdziale 3 przedstawiono analizę wymagań funkcjonalnych i niefunkcjonalnych systemu, obejmującą m.in. minimalne opóźnienia przetwarzania, odporność na awarie oraz skalowalność infrastruktury. Dokonano przeglądu istniejących rozwiązań w obszarze przetwarzania danych finansowych, w tym platform strumieniowych takich jak Apache Kafka czy NATS JetStream, oraz porównano mechanizmy składowania danych w architekturach warstwowych. Rozdział zawiera także uzasadnienie wyboru technologii wykorzystanych w projekcie.

Rozdział 4 zawiera szczegółowy projekt architektury systemu, obejmujący dekompozycję na warstwy: pozyskiwania danych, przetwarzania strumieniowego, transformacji *ETL*, składowania oraz udostępniania. Opisano strukturę baz danych MongoDB przechowujących dane operacyjne oraz konfigurację warstwy obiektowej MinIO jako pamięci zimnej. Przedstawiono przepływ danych między poszczególnymi komponentami, mechanizmy komunikacji asynchronicznej przez kolejki *JetStream* oraz zasady wersjonowania i retencji danych. Rozdział uwzględnia także schemat połączenia z interfejsem GraphQL oraz integracją z API Telegrama w celu dystrybucji powiadomień.

Rozdział 5 przedstawia szczegóły implementacyjne systemu, w tym wybrane biblioteki języka *Go* do obsługi WebSocket, deserializacji JSON oraz interakcji z bazami danych. Opisano napotkane problemy, takie jak zarządzanie ponownymi połączeniami po awariach, obsługa backpressure w strumieniach danych czy optymalizacja zapytań do MongoDB przy dużym wolumenie rekordów. Zaprezentowano także fragmenty kodu źródłowego ilustrujące kluczowe mechanizmy, w szczególności logikę archiwizacji danych oraz konstrukcję (*resolverów*) GraphQL.

Rozdział 5.3 wskazuje możliwe kierunki dalszego rozwoju systemu, obejmujące rozszerzenie o dodatkowe giełdy kryptowalut, zastosowanie algorytmów wykrywania anomalii oraz integrację z systemami monitoringu i powiadamiania, takimi jak Prometheus czy Grafana. Rozważono także możliwość wdrożenia mechanizmów uczenia maszynowego do predykcji ruchów cenowych oraz automatycznego generowania zleceń handlowych na podstawie analizy danych historycznych.

Rozdział 6 dokumentuje przeprowadzone testy manualne weryfikujące poprawność działania poszczególnych komponentów. Opisano scenariusze testowe obejmujące m.in. odbiór danych z WebSocket, transformację rekordów w procesach *ETL*, obsługę awarii komponentów oraz poprawność odpowiedzi GraphQL. Przedstawiono wyniki pomiarów opóźnień w łańcuchu przetwarzania oraz statystyki dotyczące wolumenu przetwarzanych danych.

Pracę zamyka rozdział 7, w którym podsumowano rezultaty projektu, omówiono stopień realizacji założonych celów oraz wskazano wnioski płynące z realizacji systemu. Zidentyfikowano mocne strony zaprojektowanej architektury, takie jak modularność, odporność na awarie oraz niskie koszty operacyjne, a także obszary wymagające dalszych udoskonaleń.

Rozdział 2

Przetwarzanie danych strumieniowych

Niniejszy rozdział przedstawia teoretyczne podstawy przetwarzania danych strumieniowych, ze szczególnym uwzględnieniem specyfiki rynków finansowych i kryptowalutowych. Omówiono kluczowe koncepcje, modele przetwarzania oraz wyzwania związane z obsługą ciągłych przepływów danych w czasie rzeczywistym.

2.1 Charakterystyka danych strumieniowych

Dane strumieniowe (*streaming data*) to ciągły, potencjalnie nieskończony przepływ rekordów generowanych w czasie rzeczywistym przez różnorodne źródła. W odróżnieniu od tradycyjnych zbiorów danych o skończonym rozmiarze, strumienie charakteryzują się brakiem określonego początku i końca — nowe dane napływają nieprzerwanie, a ich przetwarzanie musi odbywać się na bieżąco [6].

W kontekście rynków kryptowalutowych dane strumieniowe obejmują przede wszystkim zdarzenia transakcyjne (*trade events*), aktualizacje arkuszy zleceń (*order book updates*) oraz zmiany cen i wolumenów. Giełdy takie jak Binance generują tysiące komunikatów na sekundę dla popularnych par handlowych, co stawia wysokie wymagania przed systemami przetwarzającymi.

Kluczowe cechy danych strumieniowych z rynków finansowych to [7–9]:

- **Wysoka częstotliwość** — komunikaty napływają w odstępach milisekundowych, szczególnie w okresach zwiększonej zmienności rynkowej.
- **Zmienność natężenia** — przepływ danych nie jest stały; w momentach publikacji istotnych informacji rynkowych wolumen komunikatów może wzrosnąć wielokrotnie.
- **Krytyczność czasowa** — wartość informacji maleje wraz z upływem czasu; opóźnione dane tracą użyteczność dla algorytmów handlowych.

2.2 Modele przetwarzania danych

W literaturze i praktyce inżynierskiej wyróżnia się dwa podstawowe podejścia do przetwarzania danych: przetwarzanie wsadowe (*batch processing*) oraz przetwarzanie strumieniowe (*stream proces-*

sing). Każde z nich charakteryzuje się odmiennymi właściwościami i znajduje zastosowanie w różnych scenariuszach.

2.2.1 Przetwarzanie wsadowe

Przetwarzanie wsadowe polega na gromadzeniu danych przez określony czas, a następnie ich jednorazowym przetworzeniu jako skończonego zbioru. Model ten jest dobrze ugruntowany i szeroko stosowany w systemach analitycznych, gdzie akceptowalne są opóźnienia rzędu minut, godzin lub dni.

Typowy przepływ przetwarzania wsadowego obejmuje ekstrakcję danych ze źródeł, ich transformację zgodnie z logiką biznesową oraz załadowanie do docelowego magazynu danych (*ETL — Extract, Transform, Load*). Procesy te uruchamiane są cyklicznie, według ustalonego harmonogramu.

Zalety przetwarzania wsadowego to prostota implementacji, łatwość debugowania oraz możliwość ponownego przetworzenia całego zbioru w przypadku wykrycia błędów. Wadą jest natomiast opóźnienie między powstaniem danych a ich dostępnością dla odbiorców, co w kontekście rynków finansowych może być nieakceptowalne [10].

2.2.2 Przetwarzanie strumieniowe

Przetwarzanie strumieniowe zakłada przetwarzanie danych natychmiast po ich nadejściu, bez oczekiwania na zgromadzenie pełnego zbioru. Każdy komunikat jest analizowany indywidualnie lub w kontekście niewielkiego okna czasowego, a wyniki są udostępniane z minimalnym opóźnieniem.

W tym modelu dane przepływają przez zdefiniowany potok przetwarzania (*pipeline*), składający się z operatorów realizujących transformacje, filtrowanie, agregację czy wzbogacanie rekordów. Strumień wyjściowy może być kierowany do kolejnych etapów przetwarzania, zapisywany w bazie danych lub wykorzystywany do generowania alertów.

Przetwarzanie strumieniowe jest szczególnie istotne w kontekście danych finansowych, gdzie opóźnienia rzędu milisekund mogą decydować o rentowności strategii handlowych. Wymaga jednak bardziej złożonej infrastruktury oraz starannego zarządzania stanem przetwarzania [9].

2.2.3 Architektura Lambda i Kappa

W praktyce często stosuje się architektury hybrydowe, łączące zalety obu podejść. Architektura Lambda zakłada równoległe działanie dwóch ścieżek przetwarzania: warstwy wsadowej (*batch layer*) zapewniającej kompletność i poprawność danych historycznych oraz warstwy strumieniowej (*speed layer*) dostarczającej wyniki w czasie rzeczywistym. Warstwa udostępniania (*serving layer*) łączy wyniki z obu ścieżek, udostępniając spójny widok danych [10].

Architektura Kappa stanowi uproszczenie tego modelu poprzez eliminację dedykowanej warstwy wsadowej. W tym podejściu wszystkie dane są przetwarzane jako strumień, a przetwarzanie historyczne realizowane jest poprzez ponowne odtworzenie strumienia z trwałego magazynu zdarzeń.

Model ten jest koncepcyjnie prostszy, lecz wymaga odpowiedniej infrastruktury do przechowywania i odtwarzania historycznych strumieni [11].

2.3 Semantyka dostarczania komunikatów

Kluczowym aspektem systemów przetwarzania strumieniowego jest semantyka dostarczania komunikatów, określająca gwarancje dotyczące tego, ile razy każdy komunikat zostanie przetworzony. Wyróżnia się trzy podstawowe poziomy gwarancji:

2.3.1 At-most-once (co najwyżej raz)

W semantyce *at-most-once* komunikat może zostać utracony, lecz nigdy nie zostanie przetworzony więcej niż jeden raz. Jest to najprostsza do implementacji gwarancja, gdyż nie wymaga mechanizmów potwierdzeń ani powtórzeń. Stosowana jest w scenariuszach, gdzie sporadyczna utrata danych jest akceptowalna, na przykład w systemach monitoringu, gdzie pojedyncze brakujące pomiary nie wpływają istotnie na ogólny obraz.

W kontekście danych finansowych semantyka ta jest rzadko stosowana, ponieważ utrata nawet pojedynczej transakcji może prowadzić do niespójności w analizach lub błędnych decyzji inwestycyjnych [12].

2.3.2 At-least-once (co najmniej raz)

Semantyka *at-least-once* gwarantuje, że każdy komunikat zostanie przetworzony przynajmniej jeden raz, lecz dopuszcza możliwość duplikatów. Realizowana jest poprzez mechanizm potwierdzeń (*acknowledgements*) — komunikat pozostaje w kolejce do czasu otrzymania potwierdzenia pomyślnego przetworzenia. W przypadku awarii konsumenta lub braku potwierdzenia, komunikat jest dostarczany ponownie.

Model ten wymaga, aby konsumenci byli idempotentni, czyli wielokrotne przetworzenie tego samego komunikatu nie powinno prowadzić do niepożądanych efektów ubocznych. W praktyce idempotentność osiąga się poprzez deduplikację na podstawie unikalnych identyfikatorów komunikatów lub projektowanie operacji w sposób naturalnie idempotentny (np. ustawianie wartości zamiast inkrementacji) [12].

2.3.3 Exactly-once (dokładnie raz)

Semantyka *exactly-once* stanowi najsilniejszą gwarancję — każdy komunikat jest przetwarzany dokładnie jeden raz, bez utraty i bez duplikatów. Jest to najbardziej pożądana, lecz zarazem najtrudniejsza do osiągnięcia semantyka.

Realizacja wymaga koordynacji między producentem, brokerem i konsumentem, często z wykorzystaniem mechanizmów transakcyjnych lub rozproszonego zatwierdzania (*distributed commit*).

Ze względu na złożoność implementacji oraz wpływ na wydajność, w wielu systemach stosuje się semantykę *at-least-once* w połączeniu z idempotentnym przetwarzaniem jako praktyczny kompromis [12].

2.4 Okna czasowe w przetwarzaniu strumieniowym

Wiele operacji analitycznych wymaga agregacji danych w określonych przedziałach czasowych. W przetwarzaniu strumieniowym wykorzystuje się koncepcję okien czasowych, które grupują zdarzenia na potrzeby obliczeń zbiorczych.

2.4.1 Okna stałe (fixed windows)

Okna stałe dzielą strumień na rozłączne, następujące po sobie przedziały o jednakowej długości. Każde zdarzenie należy do dokładnie jednego okna, a wyniki agregacji są emitowane po zamknięciu każdego okna.

Przykładem zastosowania jest obliczanie średniej ceny w interwałach minutowych — każda minuta stanowi osobne okno, a po jej zakończeniu publikowany jest wynik (np. cena otwarcia, zamknięcia, maksimum, minimum). Takie agregaty stanowią podstawę wykresów świecowych (*candlestick charts*) powszechnie stosowanych w analizie technicznej [13].

2.4.2 Okna przesuwne (sliding windows)

Okna przesuwne obejmują zdarzenia z określonego przedziału czasowego, lecz w odróżnieniu od okien stałych, nakładają się na siebie. Okno przesuwa się wraz z nadejściem nowych zdarzeń, umożliwiając ciągłą aktualizację wyników agregacji.

Typowym zastosowaniem jest obliczanie średniej kroczącej (*moving average*), gdzie wynik jest aktualizowany przy każdym nowym zdarzeniu, uwzględniając ostatnie N minut lub ostatnie N transakcji. Okna przesuwne zapewniają płynniejszy obraz zmian, lecz wymagają większych zasobów obliczeniowych ze względu na nakładanie się przedziałów [13].

2.4.3 Okna sesyjne (session windows)

Okna sesyjne grupują zdarzenia na podstawie aktywności, a nie sztywnych granic czasowych. Sesja rozpoczyna się wraz z pierwszym zdarzeniem i trwa do momentu, gdy przez zdefiniowany okres nie napłyną żadne nowe zdarzenia (tzw. *gap*). Po przekroczeniu tego progu sesja jest zamykana, a kolejne zdarzenie rozpoczyna nową sesję.

W kontekście rynków finansowych okna sesyjne mogą być wykorzystywane do identyfikacji okresów wzmożonej aktywności handlowej lub grupowania serii szybkich transakcji wykonywanych przez algorytmy wysokiej częstotliwości [13].

2.5 Problemy i wyzwania

Przetwarzanie danych strumieniowych wiąże się z szeregiem wyzwań technicznych i koncepcyjnych, które wymagają starannego rozwiązania na etapie projektowania systemu.

2.5.1 Zarządzanie czasem zdarzeń

W systemach rozproszonych rozróżnia się czas zdarzenia (*event time*), czyli moment faktycznego wystąpienia zdarzenia u źródła, oraz czas przetwarzania (*processing time*), czyli moment dotarcia zdarzenia do systemu. Różnica między tymi znacznikami wynika z opóźnień sieciowych, kolejkowania oraz ewentualnych awarii.

Zdarzenia mogą docierać w kolejności innej niż chronologiczna (tzw. *out-of-order events*), co komplikuje obliczenia oparte na czasie. System musi decydować, jak długo czekać na potencjalnie opóźnione zdarzenia przed zamknięciem okna czasowego i emisją wyników. Zbyt krótkie oczekiwanie prowadzi do pominięcia opóźnionych danych, zbyt długie — do zwiększenia opóźnienia wyników.

Mechanizm *watermarks* pozwala śledzić postęp czasu zdarzeń i sygnalizować, że wszystkie zdarzenia o znaczniku czasowym mniejszym od pewnej wartości zostały już odebrane. Zdarzenia przychodzące po przekroczeniu znacznika czasu są traktowane jako spóźnione i mogą być odrzucane lub przetwarzane w specjalny sposób [13].

2.5.2 Zarządzanie stanem

Wiele operacji strumieniowych wymaga utrzymywania stanu między kolejnymi komunikatami — na przykład obliczenie średniej kroczącej wymaga pamiętania poprzednich wartości, a agregacja w oknie czasowym — gromadzenia zdarzeń do momentu zamknięcia okna.

Zarządzanie stanem w systemach rozproszonych jest wyzwaniem ze względu na konieczność zapewnienia trwałości (stan nie może zostać utracony w przypadku awarii), spójności (wszystkie repliki muszą widzieć ten sam stan) oraz wydajności (dostęp do stanu nie może stanowić wąskiego gardła).

Stosowane podejścia obejmują lokalne magazyny stanu z mechanizmem punktów kontrolnych (*checkpointing*), zewnętrzne bazy danych o niskich opóźnieniach oraz rozproszone struktury danych z replikacją [14].

2.5.3 Backpressure i kontrola przepływu

Sytuacja, w której producent generuje dane szybciej niż konsument jest w stanie je przetworzyć, nazywana jest przeciążeniem (*backpressure*). Bez odpowiednich mechanizmów kontroli przepływu system może ulec destabilizacji — kolejki rosną nieograniczenie, pamięć się wyczerpuje, a opóźnienia stale się zwiększają.

Strategie radzenia sobie z backpressure obejmują buforowanie z ograniczeniem rozmiaru (nadmiarowe komunikaty są odrzucane), adaptacyjne spowalnianie producenta (poprzez sygnalizację

zwrotną), oraz skalowanie horyzontalne konsumentów (dodawanie kolejnych instancji przetwarzających) [15].

W kontekście danych z rynków kryptowalutowych backpressure może wystąpić w okresach ekstremalnej zmienności, gdy wolumen transakcji wielokrotnie przekracza typowe wartości. System musi być zaprojektowany tak, aby degradował wydajność (zwiększając opóźnienie) zamiast tracić dane lub ulegać całkowitej awarii.

2.5.4 Skalowalność i partycjonowanie

Skalowanie systemów strumieniowych wymaga możliwości podziału strumienia na niezależne partycje, które mogą być przetwarzane równolegle. Kluczem partycjonowania jest zazwyczaj atrybut danych zapewniający równomierny rozkład obciążenia oraz zachowanie lokalności danych powiązanych logicznie [16].

W przypadku danych z giełd kryptowalut naturalnym kluczem partycjonowania jest symbol pary handlowej — wszystkie zdarzenia dotyczące danej pary (np. BTCUSDT) trafiają do tej samej partycji, co umożliwia lokalne obliczanie agregatów bez koordynacji między partycjami.

Wyzwaniem jest zapewnienie równomiernego rozkładu obciążenia, gdy popularność poszczególnych par jest zróżnicowana. Para BTC/USDT generuje znacznie więcej transakcji niż mniej popularne pary, co może prowadzić do nierównomiernego obciążenia partycji (*hot spots*) [1].

2.6 Architektura sterowana zdarzeniami

Architektura sterowana zdarzeniami (*Event-Driven Architecture*, EDA) stanowi paradygmat projektowania systemów, w którym przepływ sterowania jest determinowany przez zdarzenia — sygnały o wystąpieniu określonych stanów lub akcji w systemie.

W modelu tym komponenty systemu komunikują się poprzez publikowanie i subskrybowanie zdarzeń, bez bezpośrednich wywołań synchronicznych. Producent zdarzenia nie musi znać odbiorców, a konsumenci mogą dołączać lub odłączać się dynamicznie, co zapewnia luźne powiązanie (*loose coupling*) między modułami [17].

Centralna rola w architekturze EDA przypada brokerowi wiadomości, który odpowiada za przyjmowanie zdarzeń od producentów, ich trwałe przechowywanie oraz dostarczanie do zainteresowanych konsumentów. Broker zapewnia buforowanie w przypadku chwilowej niedostępności odbiorców oraz może realizować dodatkowe funkcje, takie jak filtrowanie, routing czy transformacja komunikatów [18].

Zalety architektury sterowanej zdarzeniami obejmują naturalną asynchroniczność (operacje nie blokują się wzajemnie), łatwość dodawania nowych konsumentów (bez modyfikacji istniejących komponentów), oraz odporność na awarie (zdarzenia są buforowane do czasu pomyślnego przetworzenia). Głównymi wadami są większa złożoność debugowania (brak liniowego przepływu sterowania) oraz konieczność zarządzania ostateczną spójnością (*eventual consistency*) [19].

2.7 Podsumowanie

Przetwarzanie danych strumieniowych z rynków finansowych wymaga starannego doboru modeli, semantyk i mechanizmów odpowiadających specyfice tego obszaru. Kluczowe znaczenie ma minimalizacja opóźnień, zapewnienie niezawodności dostarczania oraz elastyczna obsługa zmiennego obciążenia.

Architektura sterowana zdarzeniami w połączeniu z semantyką dostarczania *at-least-once* i idempotentnym przetwarzaniem stanowi praktyczny kompromis między złożonością implementacji a wymaganymi gwarancjami. Mechanizmy okien czasowych umożliwiają realizację analiz agregacyjnych, natomiast odpowiednie strategie partycjonowania i kontroli przepływu zapewniają skalowalność systemu.

W kolejnych rozdziałach przedstawiono, w jaki sposób opisane koncepcje zostały zastosowane w projektowanym systemie do przetwarzania danych z giełdy kryptowalut Binance.

Rozdział 3

Analiza wymagań

Niniejszy rozdział przedstawia analizę wymagań oraz projekt systemu do przetwarzania i archiwizacji strumieniowych danych finansowych. W pierwszej części omówiono założenia projektowe oraz kontekst technologiczny i biznesowy, w jakim system będzie funkcjonował. Następnie przedstawiono szczegółowe wymagania funkcjonalne i niefunkcjonalne, które stanowią podstawę dla dalszych decyzji architektonicznych. Rozdział zawiera również przegląd istniejących rozwiązań w obszarze przetwarzania danych finansowych oraz analizę porównawczą, która uzasadnia wybór zastosowanego podejścia.

3.1 Założenia projektu

Celem projektu jest stworzenie kompleksowego systemu do przetwarzania i archiwizacji strumieniowych danych finansowych w środowisku lokalnym (*on-premises*). System ma zapewnić nieprzerwany dostęp do danych rynkowych w czasie rzeczywistym, ich oczyszczanie, transformację oraz długoterminową archiwizację z zachowaniem pełnej historii transakcji. W odróżnieniu od rozwiązań opartych wyłącznie na zewnętrznych agregatorach danych, projektowany system zakłada autonomiczną kontrolę nad całym łańcuchem przetwarzania, co jest kluczowe w kontekście zapewnienia wiarygodności i spójności gromadzonych informacji [3].

Podstawowym założeniem jest zapewnienie ciągłości działania systemu nawet w warunkach ograniczonego dostępu do sieci zewnętrznej. Realizacja tego celu wpisuje się w paradygmat przetwarzania brzegowego (*edge computing*), który zakłada przeniesienie części logiki obliczeniowej bliżej źródła danych, minimalizując zależność od scentralizowanej infrastruktury chmurowej [4]. W kontekście danych finansowych, gdzie opóźnienia rzędu milisekund mogą mieć znaczenie dla algorytmów handlowych, lokalne przetwarzanie stanowi przewagę konkurencyjną [20].

System zakłada także wielowarstwowe przechowywanie danych (*tiered storage*), które pozwala na optymalne wykorzystanie zasobów dyskowych [5]. Dane świeże przechowywane są w warstwie operacyjnej o niskich opóźnieniach, natomiast dane historyczne migrowane są do warstwy archiwalnej charakteryzującej się niższym kosztem przechowywania, lecz większym czasem dostępu.

Takie podejście umożliwia jednoczesne spełnienie wymagań dotyczących wydajności oraz ekonomii przechowywania.

Ponadto projekt zakłada udostępnianie danych poprzez elastyczne interfejsy programistyczne, umożliwiające zarówno wykonywanie zapytań o dane bieżące z zastosowaniem złożonych filtrów, jak i pobieranie dużych zbiorów danych historycznych na potrzeby analiz retrospektywnych czy uczenia modeli predykcyjnych. System ma również realizować funkcję powiadamiania o zdarzeniach rynkowych w czasie rzeczywistym, co jest istotne w kontekście monitorowania alertów cenowych czy detekcji anomalii.

3.2 Wymagania funkcjonalne

Wymagania funkcjonalne systemu zostały podzielone na kategorie odpowiadające głównym obszarom jego działania: pobieranie, przechowywanie i przetwarzanie danych, oraz powiadamianie i udostępnianie. Każda z kategorii obejmuje zestaw atomowych wymagań, które muszą zostać spełnione przez finalną implementację.

3.2.1 Pobieranie danych

Moduł odpowiedzialny za pobieranie danych stanowi punkt wejścia całego systemu i jego poprawne działanie warunkuje jakość wszystkich dalszych etapów przetwarzania. Poniżej przedstawiono szczegółowe wymagania dla tego modułu:

- **Źródło danych** – dane pobierane są z publicznego interfejsu WebSocket giełdy Binance, która jest jedną z największych giełd kryptowalut pod względem wolumenu obrotu [21]. Wybór protokołu WebSocket podyktowany jest jego niskim opóźnieniem oraz możliwością utrzymania trwałego połączenia, co eliminuje narzut związany z wielokrotnym nawiązywaniem połączeń HTTP [22].
- **Integralność komunikatów** – każdy komunikat odebrany z interfejsu WebSocket musi zostać przekazany do dalszych etapów przetwarzania w niezmienionej formie, z zachowaniem oryginalnej struktury JSON oraz wszystkich pól metadanych. Niedopuszczalne są transformacje danych na etapie ich pobierania, które mogłyby prowadzić do utraty informacji.
- **Selekcja symboli** – system umożliwia konfigurację listy par handlowych (symboli), dla których prowadzona jest subskrypcja strumienia danych. Pozwala to na ograniczenie wolumenu przetwarzanych danych do symboli istotnych z punktu widzenia analiz.
- **Mechanizm ponownego łączenia** – w przypadku utraty połączenia WebSocket, moduł musi automatycznie podjąć próbę ponownego nawiązania połączenia.
- **Rejestracja zdarzeń** – wszystkie istotne zdarzenia związane z działaniem modułu, w tym nawiązanie i utrata połączenia, błędy interpretacji danych oraz statystyki przepustowości, muszą być rejestrowane w logach systemowych z odpowiednimi znacznikami czasowymi.

3.2.2 Przechowywanie danych

Architektura przechowywania danych opiera się na modelu dwuwarstwowym, który łączy zalety szybkiego dostępu operacyjnego z ekonomiczną archiwizacją długoterminową. Poniżej przedstawiono wymagania dla obu warstw:

- **Warstwa gorąca (*hot storage*)** – realizowana przez dokumentową bazę danych MongoDB, która przechowuje dane bieżące przez zdefiniowany okres retencji (domyślnie 24 godziny). Warstwa ta musi zapewniać niskie opóźnienia odczytu i zapisu, umożliwiając realizację zapytań analitycznych w czasie zbliżonym do rzeczywistego.
- **Warstwa zimna (*cold storage*)** – realizowana przez magazyn obiektowy MinIO, kompatybilny z protokołem Amazon S3. Warstwa ta przeznaczona jest do długoterminowego przechowywania danych historycznych z zastosowaniem kompresji (format Parquet lub gzip). Dane w warstwie zimnej organizowane są w partycje czasowe (roczne, miesięczne, dzienne) co ułatwia selektywne pobieranie zbiorów danych dla określonych przedziałów czasowych.
- **Automatyczna migracja** – system musi realizować automatyczny proces przenoszenia danych z warstwy gorącej do warstwy zimnej po upływie okresu retencji. Proces ten powinien być realizowany stopniowo, bez blokowania bieżących operacji zapisu i odczytu.
- **Spójność danych** – mechanizm migracji musi zapewniać, że żaden rekord nie zostanie utracony ani zduplikowany podczas przenoszenia między warstwami. Wymagane jest potwierdzenie zapisu w warstwie zimnej przed usunięciem danych z warstwy gorącej.

3.2.3 Przetwarzanie danych

Moduły przetwarzania danych (*ETL – Extract, Transform, Load*) odpowiadają za transformację surowych komunikatów z giełdy do postaci znormalizowanej oraz wzbogacanie ich o dodatkowe metadane analityczne:

- **Podział na zbiory surowe i oczyszczone** – system musi przechowywać zarówno surowe dane źródłowe, jak i dane po transformacji.Dane surowe stanowią podstawę wiarygodności (*source of truth*), natomiast dane oczyszczone są podstawą dla analiz.
- **Walidacja strukturalna** – każdy komunikat musi zostać poddany walidacji pod kątem zgodności ze schematem oczekiwanym dla danego typu wiadomości.
- **Idempotentność** – procesy ETL muszą być idempotentne, co oznacza, że wielokrotne przetworzenie tego samego komunikatu nie powinno prowadzić do duplikacji ani niespójności danych wynikowych [23].

3.2.4 Udostępnianie danych i powiadamianie

System musi zapewniać elastyczne mechanizmy dostępu do zgromadzonych danych oraz proaktywne powiadamianie o istotnych zdarzeniach rynkowych:

- **Interfejs API dla warstwy gorącej** – dostęp do danych bieżących realizowany jest poprzez interfejs GraphQL, który umożliwia formułowanie zapytań z wieloma filtrami oraz selekcję zwracanych atrybutów. GraphQL pozwala klientowi precyzyjnie określić strukturę odpowiedzi, minimalizując transfer zbędnych danych [24].
- **Interfejs dla warstwy zimnej** – dostęp do danych historycznych realizowany jest poprzez protokół S3, umożliwiający pobieranie całych partycji lub pojedynczych plików. Dodatkowo udostępniany jest interfejs Jupyter Notebook z bibliotekami Python do analiz ad-hoc i uczenia modeli.
- **Integracja z platformą Telegram** – dedykowany bot umożliwia użytkownikom konfigurację alertów cenowych i wolumenowych oraz otrzymywanie powiadomień o zdarzeniach rynkowych bezpośrednio na urządzenia mobilne.

3.3 Wymagania niefunkcjonalne

Oprócz wymagań funkcjonalnych, system musi spełniać szereg wymagań niefunkcjonalnych, które determinują jakość rozwiązania z punktu widzenia wydajności, niezawodności, skalowalności oraz utrzymania. Poniżej przedstawiono szczegółową specyfikację wymagań niefunkcjonalnych podzieloną na kategorie.

3.3.1 Wydajność i skalowalność

Wydajność systemu jest krytyczna z uwagi na charakter przetwarzanych danych – strumienie transakcji giełdowych mogą generować tysiące komunikatów na sekundę, a opóźnienia w przetwarzaniu mogą prowadzić do utraty informacji lub nieaktualności danych analitycznych [8]:

- **Przepustowość** – system powinien zapewniać płynną obsługę do 1 000 000 komunikatów na godzinę. W przypadku przekroczenia tej wartości, system musi gwarantować poprawne zakolejkowanie i przetworzenie wszystkich nadmiarowych komunikatów, dopuszczając przy tym zwiększone opóźnienie.
- **Opóźnienie end-to-end** – czas od odebrania komunikatu z WebSocket do jego dostępności w warstwie gorącej nie powinien przekraczać 100 ms w warunkach normalnego obciążenia.
- **Buforowanie** – system musi zapewniać buforowanie zdarzeń w warstwie komunikacji (message broker) na wypadek chwilowej niedostępności usług konsumenckich, z gwarantowanym dostarczeniem po ich wznowieniu.
- **Optymalizacja zasobów** – wykorzystanie pamięci operacyjnej i procesora powinno być proporcjonalne do obciążenia, bez wycieków pamięci (*memory leaks*) przy długotrwałym działaniu.

3.3.2 Niezawodność i odporność na awarie

Niezawodność systemu determinuje jego przydatność w środowisku produkcyjnym, gdzie ciągłość działania i integralność danych są priorytetem:

- **Odporność na awarie pojedynczych komponentów** – awaria jednego z modułów nie powinna prowadzić do utraty danych. Komunikaty muszą pozostać w buforze brokera do czasu pomyślnego przetworzenia.
- **Trwałość danych** – dane zapisane w warstwie gorącej muszą przetrwać restart usługi bazy danych. Dane w warstwie zimnej muszą być replikowane lub regularnie archiwizowane poza główny magazyn.
- **Graceful shutdown** – usługi muszą obsługiwać sygnały systemowe (SIGTERM) i realizować kontrolowane zamknięcie, zapewniając zakończenie bieżących operacji przed wyłączeniem.

3.4 Istniejące rozwiązania

W obszarze gromadzenia i przetwarzania danych rynkowych z giełd kryptowalut istnieje wiele projektów o charakterze hobbystycznym lub badawczym. Najczęściej koncentrują się one na jednym wybranym etapie łańcucha przetwarzania (np. pobieraniu danych, ich wizualizacji lub archiwizacji), rzadziej zaś oferują spójny, produkcyjny przepływ danych obejmujący pobieranie, walidację, składowanie wielowarstwowe oraz warstwę udostępniania i powiadamiania. W niniejszym podrozdziale przedstawiono cztery przykładowe rozwiązania o zbliżonej tematyce, a następnie wskazano, które ograniczenia tych projektów zostały zaadresowane w ramach systemu zaproponowanego w pracy.

3.4.1 Binance-Data-Collector-and-Visualization

Projekt *Binance-Data-Collector-and-Visualization* [25] koncentruje się na cyklicznym pobieraniu danych świecowych (*candlestick / kline*) z giełdy Binance i prezentacji ich w postaci wykresów w prostej aplikacji webowej. Zgodnie z opisem, dane mogą być zapisywane zarówno do pliku CSV, jak i do relacyjnej bazy PostgreSQL, a interfejs użytkownika pozwala na wybór instrumentu oraz interwału czasowego. Istotną cechą tego rozwiązania jest nacisk na wizualizację oraz prostotę konfiguracji, co czyni je użytecznym narzędziem demonstracyjnym.

Ograniczeniem projektu jest jednak skoncentrowanie na danych zagregowanych (świece) oraz brak kompletnego, zdarzeniowego przetwarzania danych strumieniowych w czasie rzeczywistym. W szczególności nie rozwiązuje on problemu utrzymania trwałej kolejki wiadomości, odporności na awarie konsumentów, a także długoterminowej archiwizacji w formacie zoptymalizowanym pod analitykę kolumnową. W porównaniu z tym podejściem, projekt opisany w pracy obejmuje cały potok przetwarzania danych (od WebSocket po warstwy *hot* i *cold*), umożliwiając zarówno analizę danych bieżących, jak i pracę na danych historycznych w sposób efektywny obliczeniowo.

3.4.2 binance_ticker_rust

Projekt *binance_ticker_rust* [26] stanowi aplikację napisaną w języku Rust, która łączy się z interfejsem WebSocket Binance w celu pobierania danych *ticker* w czasie rzeczywistym, a następnie zapisuje je do bazy PostgreSQL. W README podkreślono nacisk na wydajność i asynchroniczność, co jest zgodne z wymaganiami niskich opóźnień. Rozwiązanie to jest przykładem poprawnej, modularnej implementacji komponentu pobierania danych.

Jednocześnie projekt jest silnie wyspecjalizowany: obejmuje przede wszystkim pojedynczy typ danych oraz jedno miejsce składowania (PostgreSQL). Brakuje w nim warstwy udostępniania danych w czasie rzeczywistym (np. API do zapytań) oraz mechanizmów długoterminowej archiwizacji z podziałem na partycje i kompresją. Nie jest również eksponowany komponent powiadomień użytkownika o zdarzeniach rynkowych. System przedstawiony w niniejszej pracy integruje te elementy w jedną architekturę, oferując zarówno szybkie zapytania do *hot storage*, jak i masowe pobieranie danych historycznych z warstwy archiwalnej, a także moduł powiadamiania.

3.4.3 Binance Datatool (BHDS)

Projekt *Binance Datatool* [27] jest najbardziej złożony pod względem wsparcia dla danych historycznych. Jego rdzeniem jest usługa BHDS (Binance Historical Data Service), która automatyzuje pobieranie zbiorów danych historycznych z repozytorium Binance (Binance Vision) przy użyciu narzędzia Aria2 umożliwiającego pobieranie równoległe. Dane są następnie przetwarzane z wykorzystaniem biblioteki Polars i zapisywane w formacie Parquet, co jest podejściem dobrze dopasowanym do badań ilościowych. W warstwie użytkowej projekt oferuje zarówno interfejs CLI oparty o konfiguracje YAML, jak i interfejs biblioteczny.

Pomimo wysokiej jakości implementacji, rozwiązanie to koncentruje się na danych wsadowych (*batch*) i na procesie pobierania danych historycznych, pomijając kluczowe aspekty pracy ze strumieniem danych w czasie rzeczywistym. W szczególności, brak stałej subskrypcji WebSocket oraz mechanizmów natychmiastowego reagowania ogranicza zastosowanie tego narzędzia w scenariuszach monitorowania rynku. Projekt zaproponowany w pracy zakłada podejście hybrydowe łączące przetwarzanie strumieniowe ze spójną archiwizacją historyczną, przy zachowaniu kontroli nad integralnością danych i retencją.

3.4.4 binance-websocket

Projekt *binance-websocket* [28] dostarcza zestaw skryptów w języku Python służących do pobierania danych strumieniowych Spot z Binance. W repozytorium znajdują się przykładowe programy zbierające dane typu *bookTicker* oraz *depth* (migawki i aktualizacje księgi zleceń), a także zapisujące dane w postaci plików CSV lub Parquet. Zaletą projektu jest zastosowanie formatu Parquet dla części danych, co upraszcza późniejszą analizę offline.

Rozwiązanie to ma jednak charakter narzędziowy: koncentruje się na pobieraniu danych i ich zapisie do plików, nie oferując przechowywania danych, mechanizmów kolejkowania, odporności na awarie czy warstwy API. W praktyce oznacza to, że użytkownik musi samodzielnie zbudować pozostałe elementy niezbędne do utrzymania jakości danych oraz do ich wygodnego wykorzystania w aplikacjach. Projekt opisany w pracy uzupełnia te braki poprzez wprowadzenie logicznego podziału na warstwę bieżącą i archiwalną, procesy ETL, interfejs GraphQL dla zapytań oraz kanały powiadomień.

3.4.5 Wnioski z analizy porównawczej

Przedstawione projekty potwierdzają, że problem pozyskiwania i wykorzystywania danych z giełdy Binance jest szeroko podejmowany, jednak najczęściej realizowany jest w formie rozwiązań fragmentarycznych. Jedne narzędzia skupiają się na wizualizacji, inne na wydajnym pobieraniu danych historycznych, a jeszcze inne na samym przetwarzaniu strumieniowym. W każdym z przypadków widoczne są ograniczenia z perspektywy docelowego zastosowania w systemie analitycznym, tj. brak spójnego potoku danych, brak mechanizmów retencji i migracji między warstwami pamięci, brak wygodnego interfejsu zapytań oraz brak funkcji powiadomień.

System zaproponowany w niniejszej pracy stanowi próbę integracji tych rozproszonych funkcjonalności w jedno rozwiązanie, które umożliwia przetwarzanie danych w czasie rzeczywistym, ich kontrolowaną archiwizację oraz późniejsze wykorzystanie analityczne.

Rozdział 4

Projekt systemu

Niniejszy rozdział przedstawia szczegółowy projekt architektury systemu do przetwarzania i archiwizacji strumieniowych danych finansowych. System został zaprojektowany jako rozproszona architektura mikrousługowa, realizująca paradygmat przetwarzania sterowanego zdarzeniami (*Event-Driven Architecture*, EDA). Poszczególne komponenty komunikują się asynchronicznie poprzez centralnego brokera wiadomości, co zapewnia luźne powiązanie między usługami oraz odporność na chwilowe niedostępności poszczególnych modułów. Architektura ta umożliwia niezależne skalowanie, wdrażanie i aktualizowanie każdego z komponentów, co jest kluczowe w kontekście ciągłego rozwoju systemu [4].

Poniżej przedstawiono szczegółowy opis każdego z komponentów systemu wraz z opisem przepływu danych między nimi.

4.1 Serwer strumieniowy (Data Fetcher)

Serwer strumieniowy stanowi punkt wejścia systemu i odpowiada za nawiązywanie oraz utrzymywanie połączeń WebSocket z interfejsem API giełdy Binance. Komponent ten został zaimplementowany w języku Go, który charakteryzuje się niskim narzutem pamięciowym oraz natywnym wsparciem dla współbieżności poprzez mechanizm gorutyn i kanałów [29].

Serwer nasłuchuje strumieni danych dla skonfigurowanej listy par handlowych (np. BTCUSDT, ETHUSDT) i odbiera komunikaty w czasie rzeczywistym. Każdy odebrany komunikat jest walidowany pod kątem poprawności struktury JSON, uzupełniany o znacznik czasowy odbioru po stronie systemu (dla celów analizy opóźnień), a następnie publikowany do centralnego brokera wiadomości.

W przypadku utraty połączenia z giełdą, serwer realizuje algorytm ponownego łączenia. Zdarzenia związane z cyklem życia połączeń (nawiązanie, utrata, błędy) są rejestrowane w logach z odpowiednimi poziomami ważności.

Przepływ danych: `Binance API` $\rightarrow$ `Data Fetcher` $\rightarrow$ `NATS JetStream`

4.2 Broker wiadomości (NATS JetStream)

NATS JetStream pełni rolę centralnej szyny danych (*data bus*) łączącej wszystkie komponenty systemu. Jest to rozproszony system kolejkowania wiadomości charakteryzujący się wysoką przepustowością, niskimi opóźnieniami oraz wbudowanym mechanizmem trwałości (*persistence*).

Broker realizuje wzorzec publikuj-subskrybuj (*publish-subscribe*), w którym producenci (np. serwer strumieniowy) publikują komunikaty do nazwanych strumieni (*streams*), a konsumenci (np. moduł MongoDB) subskrybują te strumienie i przetwarzają komunikaty asynchronicznie. JetStream zapewnia buforowanie komunikatów w przypadku chwilowej niedostępności konsumentów, gwarantując dostarczenie po ich wznowieniu działania [30].

Kluczowe cechy konfiguracji JetStream w systemie:

- **Trwałość** – komunikaty są zapisywane na dysku, co zapewnia ich przetrwanie w przypadku restartu brokera.
- **Retencja** – konfigurowalny czas przechowywania komunikatów w strumieniu (domyślnie 24 godziny).
- **Semantyka dostarczania** – system wspiera semantykę *at-least-once*, gwarantującą dostarczenie każdego komunikatu co najmniej raz [12, 30]. Konsumenci muszą być idempotentni, aby obsłużyć potencjalne duplikaty.
- **Grupy konsumenckie** (*consumer groups*)– możliwość definiowania grup konsumentów, w ramach których każdy komunikat przetwarzany jest przez dokładnie jednego członka grupy (równoważenie obciążenia).

4.3 Warstwa gorąca (MongoDB)

MongoDB pełni rolę operacyjnej bazy danych przechowującej dane bieżące w warstwie gorącej. Wybór dokumentowej bazy danych podyktowany jest elastycznością schematu (komunikaty z giełdy mogą mieć zmienną strukturę) oraz natywnym wsparciem dla formatu JSON [31].

System zapewnia automatyczne usuwanie dokumentów po upływie okresu retencji, co odciąża warstwę operacyjną bez konieczności manualnej interwencji. Przed usunięciem, dane są przenoszone do warstwy zimnej przez dedykowany proces archiwalny.

Przepływ danych: `NATS JetStream` $\rightarrow$ `MongoDB` $\rightarrow$ `MinIO`

4.4 Moduły ETL (Extract, Transform, Load)

Moduły ETL odpowiadają za transformację surowych komunikatów z warstwy gorącej do postaci znormalizowanej oraz ich zapis do warstwy zimnej (MinIO). Architektura ETL została podzielona na dwa równoległe potoki:

1. **Potok surowych danych (Raw Writer)** – zapisuje oryginalne komunikaty bez modyfikacji, tworząc archiwum źródłowe służące jako podstawę wiarygodności (*source of truth*). Dane surowe umożliwiają debugowanie oraz ponowną transformację w przypadku zmian w logice przetwarzania.
2. **Potok danych oczyszczonych (Curated Writer)** – realizuje pełną transformację obejmującą walidację strukturalną, normalizację typów danych, wzbogacanie o metadane oraz obliczanie statystyk agregatowych.

Proces transformacji w potoku oczyszczonym obejmuje:

- Parsowanie komunikatu JSON i ekstrakcję pól kluczowych (cena, wolumen, znacznik czasowy giełdy, symbol).
- Walidację typów i zakresów wartości (np. cena > 0, wolumen ≥ 0).
- Obliczenie identyfikatora partycji czasowej (rok/miesiąc/dzień/godzina).

4.5 Warstwa zimna (MinIO Object Storage)

MinIO stanowi warstwę zimną systemu, przeznaczoną do długoterminowego przechowywania danych historycznych. Jest to magazyn obiektowy kompatybilny z protokołem Amazon S3, co umożliwia wykorzystanie szerokiego ekosystemu narzędzi analitycznych [32].

Dane w MinIO grupowane są hierarchicznie:

Partycjonowanie w stylu Hive (`key=value`) umożliwia efektywne przeszukiwanie zbiorów danych przez narzędzia takie jak Apache Spark, czy Pandas, które mogą wykorzystać strukturę katalogów do eliminacji niepotrzebnych partycji (*partition pruning*) [33].

4.6 Interfejs GraphQL

Interfejs GraphQL stanowi główny punkt dostępu do danych bieżących przechowywanych w warstwie gorącej. W odróżnieniu od tradycyjnych interfejsów REST, GraphQL umożliwia klientom precyzyjne definiowanie struktury odpowiedzi, co eliminuje problem nadmiarowego lub niedoborowego pobierania danych (*over-fetching / under-fetching*) [24].

Implementacja oparta jest na bibliotece (*gqlgen*), która generuje kod serwera Go na podstawie definicji schematu GraphQL, zapewniając typowanie statyczne i wysoką wydajność.

4.7 Serwis alertów (Telegram Bot)

Bot platformy Telegram realizuje funkcję powiadamiania użytkowników o zdarzeniach rynkowych w czasie rzeczywistym. Komponent ten subskrybuje strumień NATS JetStream i filtruje komunikaty pod kątem zdefiniowanych przez użytkowników reguł alertów.

Przepływ danych: `NATS JetStream` $\rightarrow$ `Alert Service` $\rightarrow$ `Telegram API` $\rightarrow$ `Użytkownik`

4.8 Analityka strumieniowa i historyczna

System zapewnia dwa tryby dostępu analitycznego:

1. **Analityka w czasie zbliżonym do rzeczywistego (NRT Analytics)** – realizowana na danych przepływających przez system. Obejmuje obliczanie agregatów w oknach czasowych, detekcję trendów oraz identyfikację anomalii.
2. **Analityka historyczna (Historical Analytics)** – realizowana na danych archiwalnych przechowywanych w MinIO. Dostęp zapewniany jest poprzez środowisko Jupyter Notebook z bibliotekami Python (Pandas, PyArrow, scikit-learn), umożliwiając eksploracyjną analizę danych, budowanie modeli predykcyjnych oraz generowanie raportów.

Rozdział 5

Implementacja

Niniejszy rozdział przedstawia szczegóły implementacyjne systemu do przetwarzania i archiwizacji strumieniowych danych finansowych. W pierwszej części omówiono wykorzystane technologie wraz z uzasadnieniem ich wyboru w kontekście wymagań zdefiniowanych w rozdziale analizy. Następnie przedstawiono napotkane problemy i wyzwania, które wymagały zastosowania odpowiednich wzorców projektowych i optymalizacji.

5.1 Wykorzystane technologie

Poniżej przedstawiono przegląd technologii wykorzystanych w projekcie wraz z uzasadnieniem ich wyboru.

5.1.1 Język programowania Go

Go (Golang) został wybrany jako główny język implementacji ze względu na następujące cechy [29]:

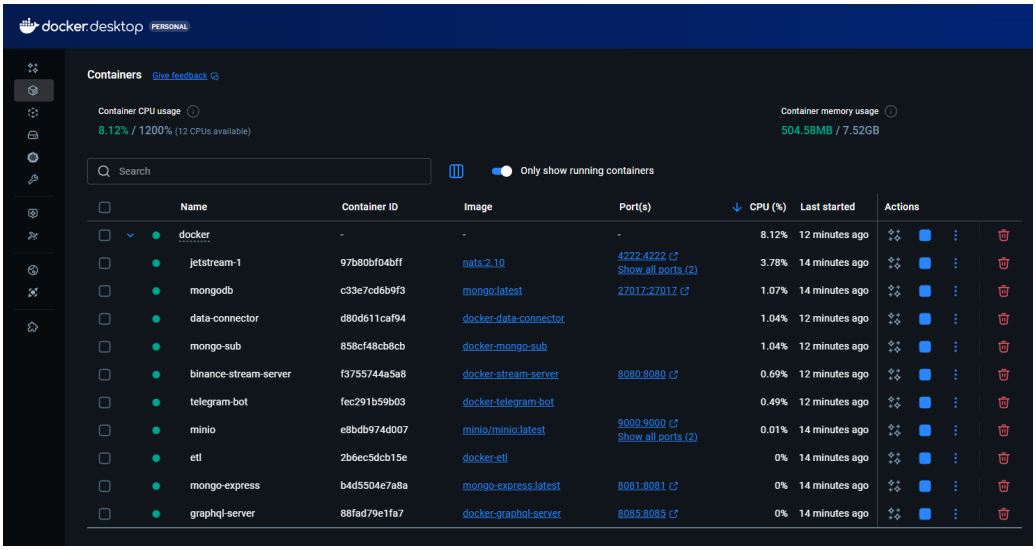
- Kompilacja do pojedynczego pliku wykonywalnego bez zewnętrznych zależności, co upraszcza dystrybucję i budowanie obrazów Docker.
- Natywne wsparcie dla współbieżności poprzez gorutyny i kanały, umożliwiające efektywną obsługę wielu połączeń WebSocket i konsumentów wiadomości.
- Niski narzut pamięciowy i szybki czas uruchamiania, istotne w środowisku kontenerowym.
- Silne typowanie statyczne z szybkim cyklem kompilacji.
- Bogate standardowe biblioteki sieciowe i HTTP.

5.1.2 Docker i Docker Compose

Konteneryzacja z wykorzystaniem Docker zapewnia [34]:

- Izolację środowiska uruchomieniowego każdego komponentu.
- Powtarzalność budowy i wdrożenia (obrazy są niemutowalne).
- Prostotę konfiguracji wielokomponentowego systemu poprzez Docker Compose.

- Łatwe skalowanie horyzontalne poprzez uruchamianie wielu instancji kontenera.



Rysunek 1. Zestawienie aktywnych usług wchodzących w skład projektowanego systemu w środowisku Docker.

5.1.3 NATS JetStream

NATS JetStream to rozproszony broker wiadomości zaprojektowany z myślą o wysokiej wydajności i niskim opóźnieniu. W porównaniu do alternatyw takich jak Apache Kafka czy RabbitMQ, JetStream charakteryzuje się prostszą konfiguracją, mniejszymi wymaganiami zasobowymi oraz natywnym wsparciem dla semantyki *at-least-once* bez konieczności zewnętrznej bazy danych [30].

NATS jest szczególnie odpowiedni dla scenariuszy wymagających niskich opóźnień, takich jak przetwarzanie danych finansowych w czasie rzeczywistym [8].

5.1.4 MongoDB

MongoDB to dokumentowa baza danych NoSQL, która zapewnia elastyczność schematu oraz wysoką wydajność zapisów [35]. W kontekście projektu, kluczowe zalety to:

- Natywna obsługa dokumentów JSON, co eliminuje konieczność mapowania obiektowo-relacyjnego.
- Bogate możliwości agregacji danych (Aggregation Framework).
- Łatwa integracja z językiem Go poprzez oficjalny sterownik.

W systemie MongoDB pełni rolę warstwy gorącej (*hot storage*) przeznaczonej do szybkiego zapisu zdarzeń oraz obsługi zapytań o najnowsze dane. Rysunki 2 oraz 3 przedstawiają strukturę kolekcji oraz przykładowe rekordy.

Mongo Express Database: data_store > Collection: BinanceTradeEvent > Document 69580c03d234863a720e4434

Editing Document: 69580c03d234863a720e4434

Back Save


```
{
    _id: ObjectId('69580c03d234863a720e4434'),
    subscriber: [
        'consumer.mongo',
        'consumer.telegrambot'
    ],
    type: 'BinanceTradeEvent',
    source: 'WebSocketStreamServer',
    eventData: {
        AggregateID: 984359054,
        LastTradeID: 3165687264,
        Price: 0.13971,
        Quantity: 76,
        Symbol: 'DOGEUSDT',
        EventTime: '2026-01-02T18:18:41.965Z',
        EventType: 'aggTrade',
        FirstTradeID: 3165687264,
        ID: '4bb4db0f-5a10-48c7-8604-ee668e3bf282',
        IsBuyerMaker: false,
        Stream: 'dogeusdt@aggTrade',
        TradeTime: '2026-01-02T18:18:41.912Z'
    },
    createdAt: '2026-01-02 18:18:43.821827315 +0000 UTC',
    event_id: 'e715fabf-3f6f-4393-8c7c-9214008e4fa6'
}
```


Back Save

Delete

Rysunek 2. Przykładowy rekord w kolekcji BinanceTradeEvent zawierający pola domenowe transakcji oraz metadane.

Mongo Express Database: data_store > Collection: NotificationEvent

Viewing Collection: NotificationEvent

New Document New Index

Simple Advanced

Key Value String Find

Delete all 1 documents retrieved

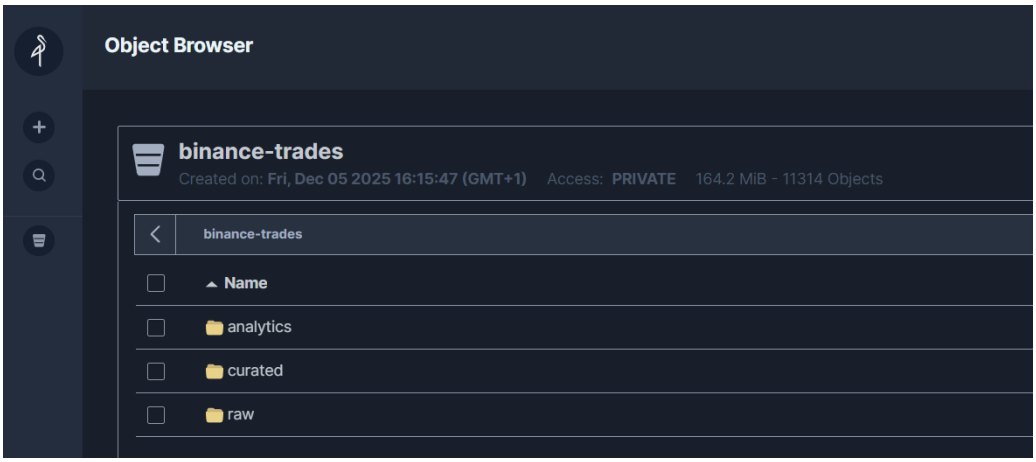
_id	type	source	eventData	createdAt	event_id
695812f4d234863a720ee04d	NotificationEvent	TelegramBot	{ "Symbol": "SOLUSDT", "IsBuyerMaker": true, "TotalPrice": 2162644.5143999998, "TradeID": "00a37674-e96b-4429-bf17-09cac9783631", "EventTime": "2026-01-02T18:48:18.702Z", "Price": 131.07, "Quantity": 16499.92 }	2026-01-02 18:48:20.915676833 +0000 UTC	758e30ed-9a4c-4ac7-a3a3-0269814ef66c

Rysunek 3. Kolekcja NotificationEvent w MongoDB przechowująca zdarzenia generowane przez system.

5.1.5 MinIO

MinIO to wysokowydajny magazyn obiektowy kompatybilny z protokołem Amazon S3. Wybór MinIO podyktowany był możliwością uruchomienia w środowisku lokalnym (*on-premises*) bez zależności od chmury publicznej, co jest kluczowe dla założeń projektu [36]. MinIO zapewnia funkcje korporacyjne takie jak wersjonowanie, szyfrowanie i replikacja, przy zachowaniu prostoty wdrożenia w kontenerze Docker [37].

W ramach systemu MinIO pełni rolę warstwy zimnej (*cold storage*) dla danych historycznych oraz warstwy analitycznej. Dane w bucketach są organizowane w sposób wspierający zarówno przetwarzanie przyrostowe, jak i zapytania wsadowe. Dla danych surowych zastosowano hierarchię katalogów według czasu (rok/miesiąc/dzień/godzina), natomiast dla danych przetworzonych i agregatów wykorzystano partycjonowanie w stylu klucz-wartość (np. symbol=DOGEUSDT/date=2025-12-05) w celu optymalizacji zapytań [33]. Rysunki od 4 oraz 5 ilustrują zawartość bucketa binance-trades w MinIO oraz przykładową zawartość pliku z agregatami czasowymi.



Rysunek 4. Widok bucketa binance-trades w MinIO z podziałem na warstwy: raw (dane surowe), curated (dane oczyszczone) oraz analytics (agregaty).

scripts/run-1764979287438301630-minute.parquet run-1764979287438301630-minute.parquet

59 rows

	symbol	date	minute_start	min_price	max_price	avg_price	volume	trade_count	notifications_triggered	first_price	last_price
1	DOGEUSDT	2025-12-05	2025-12-06 00:59:00.000	0.139	0.14	0.139	2082619	92	0	0.14	0.139
2	DOGEUSDT	2025-12-05	2025-12-06 00:58:00.000	0.14	0.14	0.14	4163876	117	0	0.14	0.14
3	DOGEUSDT	2025-12-05	2025-12-06 00:57:00.000	0.14	0.14	0.14	310026	76	0	0.14	0.14
4	DOGEUSDT	2025-12-05	2025-12-06 00:56:00.000	0.14	0.14	0.14	372752	68	0	0.14	0.14
5	DOGEUSDT	2025-12-05	2025-12-06 00:55:00.000	0.139	0.14	0.139	1296335	80	0	0.139	0.14
6	DOGEUSDT	2025-12-05	2025-12-06 00:54:00.000	0.139	0.139	0.139	1299267	76	0	0.139	0.139
7	DOGEUSDT	2025-12-05	2025-12-06 00:53:00.000	0.139	0.139	0.139	424159	68	0	0.139	0.139
8	DOGEUSDT	2025-12-05	2025-12-06 00:52:00.000	0.139	0.139	0.139	933347	67	0	0.139	0.139
9	DOGEUSDT	2025-12-05	2025-12-06 00:51:00.000	0.139	0.139	0.139	1062368	91	0	0.139	0.139
10	DOGEUSDT	2025-12-05	2025-12-06 00:50:00.000	0.139	0.139	0.139	2116718	152	0	0.139	0.139
11	DOGEUSDT	2025-12-05	2025-12-06 00:49:00.000	0.139	0.14	0.139	5847484	154	0	0.139	0.139
12	DOGEUSDT	2025-12-05	2025-12-06 00:48:00.000	0.139	0.139	0.139	1554341	87	0	0.139	0.139
13	DOGEUSDT	2025-12-05	2025-12-06 00:47:00.000	0.139	0.139	0.139	782753	92	0	0.139	0.139
14	DOGEUSDT	2025-12-05	2025-12-06 00:46:00.000	0.139	0.139	0.139	2711793	100	0	0.139	0.139
15	DOGEUSDT	2025-12-05	2025-12-06 00:45:00.000	0.139	0.139	0.139	413379	87	0	0.139	0.139
16	DOGEUSDT	2025-12-05	2025-12-06 00:44:00.000	0.139	0.139	0.139	1155289	87	0	0.139	0.139
17	DOGEUSDT	2025-12-05	2025-12-06 00:43:00.000	0.139	0.139	0.139	1295249	85	0	0.139	0.139
18	DOGEUSDT	2025-12-05	2025-12-06 00:42:00.000	0.139	0.14	0.139	840102	104	0	0.139	0.139
19	DOGEUSDT	2025-12-05	2025-12-06 00:41:00.000	0.139	0.139	0.139	578366	90	0	0.139	0.139
20	DOGEUSDT	2025-12-05	2025-12-06 00:40:00.000	0.139	0.139	0.139	1103218	89	0	0.139	0.139
21	DOGEUSDT	2025-12-05	2025-12-06 00:39:00.000	0.139	0.139	0.139	478744	91	0	0.139	0.139
22	DOGEUSDT	2025-12-05	2025-12-06 00:38:00.000	0.139	0.139	0.139	637910	123	0	0.139	0.139
23	DOGEUSDT	2025-12-05	2025-12-06 00:37:00.000	0.139	0.139	0.139	738867	99	0	0.139	0.139
24	DOGEUSDT	2025-12-05	2025-12-06 00:36:00.000	0.139	0.139	0.139	2091575	117	0	0.139	0.139
25	DOGEUSDT	2025-12-05	2025-12-06 00:35:00.000	0.139	0.139	0.139	778052	98	0	0.139	0.139
26	DOGEUSDT	2025-12-05	2025-12-06 00:34:00.000	0.139	0.139	0.139	1498679	79	0	0.139	0.139
27	DOGEUSDT	2025-12-05	2025-12-06 00:33:00.000	0.139	0.139	0.139	4617403	105	0	0.139	0.139
28	DOGEUSDT	2025-12-05	2025-12-06 00:32:00.000	0.139	0.139	0.139	728197	87	0	0.139	0.139
29	DOGEUSDT	2025-12-05	2025-12-06 00:31:00.000	0.139	0.139	0.139	508058	86	0	0.139	0.139
30	DOGEUSDT	2025-12-05	2025-12-06 00:30:00.000	0.139	0.139	0.139	1499723	106	0	0.139	0.139
31	DOGEUSDT	2025-12-05	2025-12-06 00:29:00.000	0.139	0.139	0.139	931784	80	0	0.139	0.139
32	DOGEUSDT	2025-12-05	2025-12-06 00:28:00.000	0.139	0.139	0.139	3534165	122	0	0.139	0.139
33	DOGEUSDT	2025-12-05	2025-12-06 00:27:00.000	0.139	0.139	0.139	2064738	99	0	0.139	0.139
34	DOGEUSDT	2025-12-05	2025-12-06 00:26:00.000	0.139	0.139	0.139	1459637	105	0	0.139	0.139

run-1764979287438301630-minute.parquet Data

Rysunek 5. Przykładowa zawartość pliku z agregatami minutowymi w warstwie analytics.

5.1.6 GraphQL i gqlgen

GraphQL to język zapytań do interfejsów API, który umożliwia klientom precyzyjne definiowanie struktury odpowiedzi. Biblioteka gqlgen generuje typowany kod Go na podstawie schematu GraphQL, zapewniając spójność typów w czasie kompilacji oraz wysoką wydajność dzięki unikaniu refleksji w czasie wykonania [38].

Schemat GraphQL zapisywany jest w postaci SDL (*Schema Definition Language*), czyli deklaratywnej definicji obiektów domenowych, typów wejściowych (*input*) oraz dostępnych operacji (*query*). W SDL znak wykrzyknika oznacza pole wymagane (*non-null*), tzn. serwer gwarantuje, że wartość nie będzie pusta. Z kolei nawiasy kwadratowe oznaczają listę elementów [39].

Poniższy zrzut ekranu (Rysunek 6) przedstawia przykładowe zapytanie GraphQL, które ilustruje elastyczność API w zakresie doboru pól oraz warunków filtrowania.

```
1  query allFields{
2    trades(filter: {
3      Symbol: "DOGEUSDT"
4      Limit: 3
5      SortField2: "TradeTime"
6      SortDir2: DESC
7    }) {
8      Symbol
9      Price
10     TradeTime
11   }
12 }
13
14
```

```
{
  "data": {
    "trades": [
      {
        "Symbol": "DOGEUSDT",
        "Price": 0.13875,
        "TradeTime": "2026-01-02T18:46:19.462Z"
      },
      {
        "Symbol": "DOGEUSDT",
        "Price": 0.13875,
        "TradeTime": "2026-01-02T18:46:18.437Z"
      },
      {
        "Symbol": "DOGEUSDT",
        "Price": 0.13875,
        "TradeTime": "2026-01-02T18:46:18.131Z"
      }
    ]
  }
}
```

Rysunek 6. Przykładowe zapytanie: trzy najnowsze zdarzenia dla symbolu DOGEUSDT.

5.1.7 Modele danych w języku Go

Warstwa modeli w języku Go opisuje struktury danych wykorzystywane na kolejnych etapach potoku: od danych wejściowych z WebSocket, przez zdarzenia zapisywane w bazie, aż po dane przygotowane do archiwizacji w magazynie obiektowym. Jawne typy struktur stanowią kontrakt między mikroserwisami oraz ograniczają liczbę błędów integracyjnych dzięki weryfikacji typów w czasie kompilacji. Rysunek 7 przedstawia strukturę (*BinanceTradeData*) wykorzystywaną do reprezentacji danych odebranych z WebSocket.

```
Implement interface
5   type BinanceTradeData struct {  amdla
6       ID           string    `bson:"ID" json:"ID"`
7       Stream       string    `bson:"Stream" json:"Stream"`
8       EventType    string    `bson:"EventType" json:"EventType"`
9       EventTime    time.Time `bson:"EventTime" json:"EventTime"`
10      AggregateID  int64     `bson:"AggregateID" json:"AggregateID"`
11      Symbol       string    `bson:"Symbol" json:"Symbol"`
12      Price        float64   `bson:"Price" json:"Price"`
13      Quantity     float64   `bson:"Quantity" json:"Quantity"`
14      FirstTradeID int64     `bson:"FirstTradeID" json:"FirstTradeID"`
15      LastTradeID  int64     `bson:"LastTradeID" json:"LastTradeID"`
16      TradeTime    time.Time `bson:"TradeTime" json:"TradeTime"`
17      IsBuyerMaker bool      `bson:"IsBuyerMaker" json:"IsBuyerMaker"`
18  }
```

Rysunek 7. Struktura BinanceTradeData — podstawowy model danych transakcyjnych pobierany z WebSocket.

5.1.8 Format Parquet

Apache Parquet to kolumnowy format przechowywania danych, zoptymalizowany pod kątem analitycznych obciążeń odczytu. Kluczowe zalety [40]:

- Efektywna kompresja kolumnowa (często 10x redukcja rozmiaru względem JSON).
- Szybkie odczyty selektywne (tylko wybrane kolumny są dekompresowane).

- Wsparcie dla złożonych schematów z zagnieżdżonymi strukturami.
- Szeroka kompatybilność z narzędziami analitycznymi (Pandas, Spark, DuckDB).

5.1.9 Viper

Viper to biblioteka Go do zarządzania konfiguracją, wspierająca wiele źródeł z możliwością hierarchicznego nadpisywania. Umożliwia elastyczne konfigurowanie mikroserwisów bez konieczności rekompilacji [41].

5.1.10 Jupyter Notebook i Python

Jupyter Notebook służy jako interfejs do eksploracyjnej analizy danych historycznych przechowywanych w MinIO. Środowisko Python z bibliotekami Pandas, PyArrow i scikit-learn umożliwia [42]:

- Interaktywną eksplorację zbiorów danych.
- Wizualizację trendów i anomalii.
- Prototypowanie modeli uczenia maszynowego.
- Generowanie raportów analitycznych.

5.2 Problemy i wyzwania

Realizacja systemu o architekturze rozproszonej wiąże się z szeregiem wyzwań projektowych i implementacyjnych, które wymagają świadomych kompromisów oraz zastosowania odpowiednich wzorców i praktyk inżynierskich. Poniżej przedstawiono kluczowe problemy zidentyfikowane w trakcie projektowania i implementacji systemu.

5.2.1 Złożoność architektury mikrousługowej

Architektura mikrousługowa, pomimo licznych zalet takich jak niezależne skalowanie i wdrażanie, wprowadza znaczącą złożoność operacyjną. Każdy mikroserwis wymaga własnej konfiguracji, zarządzania zależnościami, obsługi błędów oraz monitorowania. Liczba potencjalnych punktów awarii rośnie proporcjonalnie do liczby komponentów, co wymaga wdrożenia strategii odporności (*resilience patterns*).

Komunikacja między serwisami poprzez sieć wprowadza dodatkowe źródła opóźnień i błędów (timeouty, utrata połączenia, przeciążenie), które muszą być obsługiwane na poziomie aplikacji. W tradycyjnych systemach monolitycznych, wywołania między modułami realizowane są w ramach jednego procesu, co eliminuje tę klasę problemów [43].

5.2.2 Debugowanie i śledzenie rozproszone

Diagnostyka problemów w systemie rozproszonym jest znacząco bardziej złożona niż w aplikacji monolitycznej. Pojedyncze żądanie użytkownika może przepływać przez wiele usług, a logi każdej z nich

muszą zostać skorelowane w celu odtworzenia pełnego obrazu sytuacji. Wymaga to implementacji mechanizmów propagacji kontekstu (np. identyfikatorów korelacji) oraz centralizacji logów.

Dodatkowo, błędy związane z kolejnością zdarzeń są trudniejsze do reprodukcji w środowisku rozproszonym, gdzie czas wykonania poszczególnych operacji zależy od wielu czynników zewnętrznych [44].

5.2.3 Przetwarzanie sterowane zdarzeniami

Paradygmat *Event-Driven Architecture* wymaga odmiennego podejścia do projektowania niż tradycyjne systemy oparte na synchronicznych wywołaniach. Należy precyzyjnie zdefiniować strukturę zdarzeń (kontrakty), zapewnić wersjonowanie schematów oraz obsłużyć scenariusze takie jak zdarzenia niespójne czasowo (*out-of-order events*) czy zduplikowane komunikaty [12].

5.2.4 Zarządzanie wieloma kontenerami

System składa się z wielu kontenerów Docker, których uruchomienie i koordynacja wymaga odpowiedniej konfiguracji orkiestracji. Proces uruchamiania pełnego środowiska deweloperskiego jest czasochłonny (kilka minut), co wydłuża cykl debugowania — nawet drobna literówka w kodzie wymaga przebudowania obrazu i restartu kontenera.

Ponadto, duża liczba kontenerów i wolumen przetwarzanych danych przekładają się na znaczące wymagania zasobowe. Optymalizacja wykorzystania pamięci i procesora jest niezbędna do uruchomienia systemu na typowej stacji roboczej dewelopera.

5.2.5 Jednoczesny dostęp do danych historycznych i bieżących

Specyficznym wyzwaniem jest zapewnienie jednoczesnego dostępu do danych w dwóch warstwach o odmiennych charakterystykach. Warstwa gorąca (MongoDB) optymalizowana jest pod kątem szybkich zapisów i zapytań punktowych, natomiast warstwa zimna (MinIO) przeznaczona jest do sekwencyjnego odczytu dużych zbiorów danych.

5.2.6 Projektowanie elastycznego API

Zaprojektowanie interfejsu GraphQL umożliwiającego elastyczne zapytania z wieloma filtrami wymagało głębokiego zrozumienia specyfiki tego standardu. GraphQL różni się znacząco od REST pod względem modelowania danych, obsługi błędów oraz optymalizacji wydajności [38].

Dodatkowo, biblioteka gqlgen wymaga zdefiniowania schematu w języku SDL (*Schema Definition Language*), a następnie implementacji (*resolverów*) — funkcji odpowiedzialnych za pobieranie danych dla poszczególnych pól [39].

5.2.7 Optymalizacja procesu archiwizacji

Efektywne przenoszenie danych między warstwami wymaga starannej optymalizacji pod kątem wykorzystania zasobów sieciowych i dyskowych. Proces musi być realizowany przyrostowo, bez blokowania bieżących operacji, z mechanizmem wznawiania w przypadku przerwania.

Kompresja danych do formatu Parquet jest operacją kosztowną obliczeniowo, dlatego należy wyważyć stopień kompresji (a co za tym idzie oszczędność miejsca) z czasem wykonania procesu archiwizacji.

5.3 Możliwości dalszego rozwoju

Wdrożony system stanowi działający *proof of concept* architektury do przetwarzania i archiwizacji danych rynkowych w czasie rzeczywistym w środowisku *on-premises*. Z uwagi na mikrousługowy charakter rozwiązania oraz silnie operacyjny kontekst przetwarzania strumieniowego, istnieje szereg naturalnych kierunków rozwoju, które pozwoliłyby zwiększyć niezawodność, ułatwić diagnozę problemów oraz rozszerzyć funkcjonalność analityczną. Poniżej przedstawiono wybrane propozycje.

5.3.1 Centralizacja logów

W obecnej postaci każdy mikroserwis zapisuje logi niezależnie, co utrudnia szybkie odtworzenie przebiegu zdarzeń w sytuacji awarii lub rozbieżności danych. Wprowadzenie centralizacji logów (np. poprzez standaryzację logów w formacie JSON, dodanie identyfikatora korelacji oraz wysyłkę do wspólnego backendu typu Loki/ELK) umożliwiłoby analizę całego systemu w jednym miejscu. Taka zmiana skraca czas diagnostyki oraz ułatwia analizę zależności między komponentami.

5.3.2 Metryki i dashboardy (Prometheus/Grafana)

Kolejnym krokiem jest ekspozycja metryk technicznych i domenowych, takich jak: opóźnienie od odbioru zdarzenia do zapisania w bazie, procent błędów walidacji, opóźnienie konsumentów JetStream, czasy odpowiedzi API GraphQL czy wydajność procesów archiwizacji. Wysyłanie tych metryk do systemu typu Prometheus oraz budowa dashboardów w Grafanie pozwoliłaby nie tylko monitorować bieżący stan, ale również planować zasoby na podstawie trendów i wykrywać degradację wydajności zanim stanie się odczuwalna dla użytkownika [45].

5.3.3 Śledzenie rozproszone (distributed tracing)

W systemie sterowanym zdarzeniami pojedyncze zdarzenie przechodzi przez wiele komponentów, co utrudnia śledzenie jego przebiegu. Wdrożenie śledzenia rozproszonego (np. OpenTelemetry) wraz z propagacją kontekstu (`trace_id`) między mikroserwisami pozwoliłoby na wizualizację pełnej ścieżki przetwarzania i precyzyjne wskazanie wąskich gardeł [44]. Jest to szczególnie istotne przy skalowaniu systemu oraz przy analizie incydentów trudnych do odtworzenia.

5.3.4 Alerty operacyjne i powiadomienia o stanie systemu

W projekcie istnieje bot Telegram wykorzystywany do powiadomień o zdarzeniach rynkowych. Naturalnym rozszerzeniem jest dodanie warstwy alertów operacyjnych, które informowałyby o problemach działania systemu: braku nowych danych, wysokim poziomie błędów, rosnącym opóźnieniu konsumentów, przekroczeniu progów zużycia zasobów czy nieudanych migracjach danych do MinIO. Alerty oparte o metryki i logi pozwoliłyby reagować szybciej i skrócić czas przestojów.

5.3.5 Rozszerzenie API: dostęp do danych z MinIO oraz federacja GraphQL

Obecne API GraphQL udostępnia dane z warstwy gorącej (MongoDB). Dodanie zapytań do danych historycznych i analitycznych przechowywanych w MinIO umożliwiłoby obsługę szerszego zakresu przypadków użycia, takich jak pobieranie agregatów, dłuższych okien czasowych oraz danych do treningu modeli.

W praktyce wygodnym podejściem jest zastosowanie federacji GraphQL, czyli mechanizmu pozwalającego łączyć wiele niezależnych schematów i serwerów GraphQL w jeden spójny endpoint. Federacja działa jak *GraphQL dla endpointów GraphQL*: klient widzi jedno API pod jednym adresem, natomiast bramka *gateway* rozdziela zapytania do właściwych podserwisów i składa odpowiedź w całość [46]. Takie rozwiązanie upraszcza rozwój i wdrażanie, ponieważ poszczególne części API mogą ewoluować niezależnie.

5.3.6 Bardziej zaawansowany moduł analityczny

Aktualna warstwa analityczna ma charakter demonstracyjny i koncentruje się na podstawowej agregacji. Kolejnym krokiem mogłoby być rozszerzenie zakresu obliczeń o cechy realnie wykorzystywane w analizie ilościowej, takie jak zmienność w oknach kroczących, miary trendu i momentum, wykrywanie anomalii czy proste sygnały zdarzeniowe.

5.3.7 Modele predykcyjne i pipeline uczenia maszynowego

System dostarcza obecnie dane oczyszczone i zagregowane, co stanowi dobry fundament do budowy modeli predykcyjnych. Dalszy rozwój mógłby obejmować przygotowanie pełnego potoku ML: definicję etykiet, walidację czasową, rejestrowanie eksperymentów oraz wersjonowanie modeli. W wariancie produkcyjnym można rozważyć osobny serwis publikujący predykcje jako kolejne zdarzenia w NATS, co pozwoliłoby wykorzystywać je w powiadomieniach lub dalszych procesach ETL.

5.3.8 Dalsza optymalizacja i skalowanie do wszystkich symboli

Aktualnie system obsługuje ograniczoną liczbę symboli, co upraszcza przetwarzanie i zmniejsza wymagania zasobowe. Skalowanie do pełnej listy symboli dostępnych na giełdzie wymagałoby poprawy mechanizmów *backpressure*, fragmentacji strumieni i konsumentów (np. podział po

symbolu lub klasie instrumentów), grupowania zapisów do baz oraz lepszego ograniczania kosztów obliczeniowych w ETL. Taki kierunek rozwoju jest istotny, jeśli celem systemu miałoby być pełne odwzorowanie rynku w czasie rzeczywistym.

Rozdział 6

Weryfikacja rozwiązania

W celu weryfikacji poprawności działania zaprojektowanego systemu przeprowadzono serię testów manualnych obejmujących wszystkie kluczowe komponenty architektury. Testy miały na celu potwierdzenie, że każdy z mikroserwisów realizuje przypisane mu funkcje zgodnie z założeniami przedstawionymi w rozdziałach 3 i 4.

Scenariusze testowe skupiały się na weryfikacji następujących aspektów systemu:

- Poprawność połączenia z zewnętrznymi źródłami danych (WebSocket Binance) oraz odbierania strumieniowych zdarzeń rynkowych.
- Funkcjonowanie magistrali komunikacyjnej NATS JetStream, w tym publikowanie i subskrybowanie zdarzeń przez poszczególne komponenty.
- Działanie systemu powiadomień Telegram oraz modułu zapisu do bazy MongoDB.
- Działanie mechanizmu filtrowania powiadomień w zależności od parametrów zdarzeń.
- Poprawność zapytań GraphQL z różnymi filtrami, zarówno zwracającymi wyniki, jak i pustymi zbiorami danych.
- Proces ETL oraz mechanizm *tiered storage*, obejmujący przenoszenie danych z warstwy gorącej (MongoDB) do warstwy zimnej (MinIO) oraz tworzenie zbiorów danych w formacie Parquet.

Każdy test został przeprowadzony w środowisku Docker z wykorzystaniem obrazów zbudowanych na podstawie kodu źródłowego projektu. Wszystkie scenariusze zakończyły się wynikiem pozytywnym, potwierdzając tym samym zgodność implementacji z założeniami projektowymi.

Poniżej przedstawiono szczegółowe opisy poszczególnych scenariuszy testowych.

6.1 Test T1: Odbiór danych z Binance WebSocket

Test weryfikuje poprawność połączenia serwera strumieniowego z API Binance oraz odbierania komunikatów WebSocket zawierających zdarzenia typu *aggTrade*.

Tablica 1. Scenariusz testowy T1: Odbiór danych z Binance WebSocket.

Pole	**Treść**
Nazwa scenariusza	Odbiór danych rynkowych z Binance WebSocket
Cel testu	Weryfikacja poprawności nawiązania połączenia WebSocket z API Binance oraz odbierania strumieniowych zdarzeń transakcyjnych dla wybranych par handlowych
Parametry testu	Lista symboli: *BTCUSDT, ETHUSDT*; URL WebSocket: *wss://fstream.binance.com/stream?streams= btcusdt@aggTrade/ethusdt@aggTrade*
Kroki testowe	1. Uruchomienie kontenera *stream_server*. 2. Nawiązanie połączenia z endpointem Binance WebSocket. 3. Nasłuchiwanie wiadomości przychodzących. 4. Analiza logów aplikacji w celu potwierdzenia odbioru zdarzeń.
Oczekiwany rezultat	Aplikacja nawiązuje połączenie WebSocket, odbiera komunikaty w formacie JSON zawierające pola: *stream*, *data* (z zagnieżdżonymi polami *e, E, s, p, q, T, m*), oraz loguje odbiór zdarzeń.
Rzeczywisty rezultat	Połączenie zostało nawiązane poprawnie. Aplikacja otrzymała komunikaty JSON zgodne z dokumentacją Binance, w logach pojawiły się wpisy potwierdzające odbiór zdarzeń typu *aggTrade*.

Test potwierdził, że serwer strumieniowy poprawnie nawiązuje połączenie z API Binance oraz odbiera dane rynkowe w czasie rzeczywistym. Otrzymane komunikaty zawierały wszystkie wymagane pola domenowe (symbol, cena, wolumen, znacznik czasowy), co umożliwiło dalsze przetwarzanie zdarzeń przez system.

6.2 Test T2: Publikowanie zdarzeń do NATS JetStream

Test weryfikuje poprawność publikowania odebranych zdarzeń z Binance do magistrali komunikacyjnej NATS JetStream oraz ich dostępność w kolejce dla innych konsumentów.

Tablica 2. Scenariusz testowy T2: Publikowanie zdarzeń do NATS JetStream.

Pole	**Treść**
Nazwa scenariusza	Publikowanie zdarzeń do kolejki NATS JetStream
Cel testu	Weryfikacja poprawności publikowania zdarzeń typu *BinanceTradeEvent* do strumienia NATS oraz ich trwałego zapisu w kolejce
Parametry testu	Stream: *EVENTS*, Subject: *input.events*, Typ zdarzenia: *BinanceTradeEvent* zawierający pola: *ID, Subscribers, Type, Source, EventData, CreatedAt*
Kroki testowe	1. Uruchomienie serwera strumieniowego oraz NATS JetStream. 2. Odebranie komunikatu z Binance WebSocket. 3. Opakowanie danych w strukturę *Event* i publikacja do kolejki. 4. Weryfikacja obecności zdarzenia w streamie za pomocą logów aplikacji.
Oczekiwany rezultat	Zdarzenie zostaje poprawnie opublikowane do strumienia *EVENTS* na subject *input.events*. W logach pojawia się wpis potwierdzający publikację wraz z identyfikatorem zdarzenia. Zdarzenie jest dostępne dla konsumentów subskrybujących dany strumień.
Rzeczywisty rezultat	Zdarzenie zostało pomyślnie opublikowane do NATS JetStream. Logi aplikacji potwierdziły publikację z odpowiednim identyfikatorem UUID. Zdarzenie znalazło się w kolejce i jest dostępne dla konsumentów.

Test potwierdził poprawne działanie mechanizmu publikowania zdarzeń do magistrali NATS JetStream. Serwer strumieniowy przekształca surowe komunikaty z WebSocket w strukturę *Event* zawierającą metadane (typ, źródło, lista subskrybentów), a następnie publikuje je do kolejki.

6.3 Test T3: Subskrybowanie zdarzeń przez moduł MongoDB

Test weryfikuje poprawność subskrybowania zdarzeń z kolejki NATS JetStream przez moduł MongoDB oraz zapisywania ich do odpowiednich kolekcji.

Tablica 3. Scenariusz testowy T3: Subskrybowanie zdarzeń przez moduł MongoDB.

Pole	**Treść**
Nazwa scenariusza	Odbiór zdarzeń przez konsument MongoDB
Cel testu	Weryfikacja poprawności subskrybowania zdarzeń z kolejki NATS oraz zapisywania ich do bazy MongoDB w odpowiedniej kolekcji
Parametry testu	Subject: *consumer.mongo*, Durable: *mongo-subscriber-durable*, Typ zdarzenia: *BinanceTradeEvent*, Kolekcja docelowa: *BinanceTradeEvent*
Kroki testowe	1. Uruchomienie kontenerów NATS oraz MongoDB. 2. Publikacja zdarzenia do kolejki NATS na subject *consumer.mongo*. 3. Oczekiwanie na przetworzenie zdarzenia przez konsumenta. 4. Weryfikacja obecności rekordu w bazie MongoDB w kolekcji *BinanceTradeEvent*.
Oczekiwany rezultat	Konsument MongoDB odbiera zdarzenie z kolejki NATS, wyodrębnia pola: *event_id, subscriber, type, source, eventData, createdAt*, tworzy dokument BSON i zapisuje go do kolekcji. W logach pojawia się wpis potwierdzający zapis z identyfikatorem zdarzenia.
Rzeczywisty rezultat	Zdarzenie zostało odebrane i przetworzone poprawnie. Dokument z odpowiednimi polami został zapisany do kolekcji *BinanceTradeEvent*. Logi potwierdziły operację z identyfikatorem zdarzenia.

Test potwierdził poprawne działanie konsumenta MongoDB jako subskrybenta zdarzeń z magistrali NATS. Moduł poprawnie odbiera komunikaty, deserializuje je z formatu JSON do struktury *Event*, konwertuje na dokumenty BSON oraz zapisuje do odpowiednich kolekcji na podstawie pola *type*.

6.4 Test T4: Subskrybowanie zdarzeń przez moduł Telegram Bot

Test weryfikuje poprawność subskrybowania zdarzeń z kolejki NATS JetStream przez moduł Telegram Bot oraz przetwarzania komunikatów w celu wysyłania powiadomień.

Tablica 4. Scenariusz testowy T4: Subskrybowanie zdarzeń przez moduł Telegram Bot.

Pole	**Treść**
Nazwa scenariusza	Odbiór zdarzeń przez konsument Telegram Bot
Cel testu	Weryfikacja poprawności subskrybowania zdarzeń z kolejki NATS przez moduł Telegram Bot oraz przetwarzania komunikatów
Parametry testu	Subject: *consumer.telegrambot*, Durable: *telegram-bot-durable*, Typ zdarzenia: *BinanceTradeEvent*
Kroki testowe	1. Uruchomienie kontenerów NATS oraz *telegram_bot*. 2. Publikacja zdarzenia do kolejki NATS na subject *consumer.telegrambot*. 3. Oczekiwanie na przetworzenie zdarzenia przez konsument. 4. Analiza logów aplikacji w celu potwierdzenia odbioru i przetworzenia zdarzenia.
Oczekiwany rezultat	Konsument Telegram Bot odbiera zdarzenie z kolejki NATS, deserializuje je do struktury *Event*, wyodrębnia dane *BinanceTradeData* i przetwarza je zgodnie z logiką biznesową. W logach pojawia się wpis potwierdzający odbiór zdarzenia.
Rzeczywisty rezultat	Zdarzenie zostało odebrane i przetworzone poprawnie. Moduł Telegram Bot zdeserializował komunikat, wyodrębnił dane transakcji. Logi potwierdziły operację z odpowiednim identyfikatorem zdarzenia.

Test potwierdził poprawne działanie konsumenta Telegram Bot jako subskrybenta zdarzeń z magistrali NATS. Moduł poprawnie odbiera komunikaty oraz deserializuje zagnieżdżone dane typu *BinanceTradeData* z pola *EventData*, przygotowując je do dalszego przetwarzania przez mechanizm filtrowania.

6.5 Test T5: Wysyłanie powiadomienia Telegram – filtr spełniony

Test weryfikuje poprawność działania mechanizmu filtrowania zdarzeń oraz wysyłania powiadomień przez Telegram Bot w przypadku spełnienia warunku filtra.

Tablica 5. Scenariusz testowy T5: Wysyłanie powiadomienia Telegram – filtr spełniony.

Pole	**Treść**
Nazwa scenariusza	Wysyłanie powiadomienia przy spełnieniu warunku filtra
Cel testu	Weryfikacja poprawności działania mechanizmu filtrowania oraz wysyłania powiadomień do Telegram w przypadku, gdy wartość transakcji przekracza ustalony próg
Parametry testu	Zdarzenie *BinanceTradeEvent* z polami: *Price = 50000.0*, *Quantity = 50.0*, wartość całkowita transakcji: 2 500 000 USD, próg filtra: 2 000 000 USD
Kroki testowe	1. Publikacja zdarzenia do kolejki NATS na subject *consumer.telegrambot*. 2. Oczekiwanie na przetworzenie zdarzenia przez Telegram Bot. 3. Weryfikacja warunku filtra (cena x ilość > 2 000 000). 4. Sprawdzenie logów potwierdzających wysłanie powiadomienia. 5. Weryfikacja otrzymania wiadomości na czacie Telegram.
Oczekiwany rezultat	Moduł Telegram Bot oblicza wartość całkowitą transakcji (50000.0 x 50.0 = 2 500 000), porównuje ją z progiem 2 000 000 USD, stwierdza spełnienie warunku i wysyła powiadomienie do Telegram API zawierające symbol, cenę, ilość oraz sformatowaną wartość całkowitą. W logach pojawia się wpis o wysłaniu alertu.
Rzeczywisty rezultat	Warunek filtra został spełniony. Telegram Bot wysłał powiadomienie z odpowiednimi danymi transakcji. Wiadomość została dostarczona na czat Telegram z prawidłowo sformatowaną wartością całkowitą (2 500 000.00). Logi potwierdziły operację.

Test potwierdził poprawne działanie mechanizmu filtrowania zdarzeń oraz integracji z Telegram API. System poprawnie oblicza wartość całkowitą transakcji, porównuje ją z progiem zdefiniowanym w funkcji *ValidateTrade* oraz wysyła powiadomienie w formacie Markdown z czytelnie sformatowaną kwotą.

6.6 Test T6: Brak powiadomienia Telegram – filtr niespełniony

Test weryfikuje poprawność działania mechanizmu filtrowania w przypadku, gdy zdarzenie nie spełnia warunku wysyłania powiadomienia.

Tablica 6. Scenariusz testowy T6: Brak powiadomienia Telegram – filtr niespełniony.

Pole	**Treść**
Nazwa scenariusza	Brak powiadomienia przy niespełnieniu warunku filtra
Cel testu	Weryfikacja poprawności działania mechanizmu filtrowania oraz potwierdzenie braku wysłania powiadomienia do Telegram w przypadku, gdy wartość transakcji nie przekracza ustalonego progu
Parametry testu	Zdarzenie *BinanceTradeEvent* z polami: *Price = 30000.0*, *Quantity = 10.0*, wartość całkowita transakcji: 300 000 USD, próg filtra: 2 000 000 USD
Kroki testowe	1. Publikacja zdarzenia do kolejki NATS na subject *consumer.telegrambot*. 2. Oczekiwanie na przetworzenie zdarzenia przez Telegram Bot. 3. Weryfikacja warunku filtra (cena x ilość > 2 000 000). 4. Sprawdzenie logów potwierdzających odrzucenie zdarzenia bez wysyłania powiadomienia. 5. Potwierdzenie braku nowej wiadomości na czacie Telegram.
Oczekiwany rezultat	Moduł Telegram Bot oblicza wartość całkowitą transakcji (30000.0 x 10.0 = 300 000), porównuje ją z progiem 2 000 000 USD, stwierdza brak spełnienia warunku i pomija wysłanie powiadomienia. Zdarzenie zostaje potwierdzone (Ack) bez dalszego przetwarzania. Brak wpisu w logach o wysłaniu alertu.
Rzeczywisty rezultat	Warunek filtra nie został spełniony. Telegram Bot przetworzył zdarzenie, obliczył wartość transakcji, stwierdził brak spełnienia progu i nie wysłał powiadomienia. Zdarzenie zostało potwierdzone poprawnie. Brak nowej wiadomości na czacie Telegram.

Test potwierdził poprawne działanie logiki filtrowania zdarzeń. System nie generuje nadmiarowych powiadomień dla transakcji o wartości poniżej ustalonego progu, co zapobiega zaspamowaniu użytkownika alertami o małej istotności oraz redukuje liczbę wywołań Telegram API.

6.7 Test T7: Zapytanie GraphQL z filtrami zwracającymi wyniki

Test weryfikuje poprawność działania interfejsu GraphQL oraz mechanizmu filtrowania danych zapisanych w MongoDB z wykorzystaniem warunków zawsze spełnionych.

Tablica 7. Scenariusz testowy T7: Zapytanie GraphQL z filtrami zwracającymi wyniki.

Pole	**Treść**
Nazwa scenariusza	Zapytanie GraphQL z filtrami zwracającymi rekordy
Cel testu	Weryfikacja poprawności wykonywania zapytań GraphQL z wykorzystaniem filtrów czasowych oraz potwierdzenie zwracania poprawnych danych z bazy MongoDB
Parametry testu	Zapytanie GraphQL typu *trades* z filtrem: *eventTime.after > "2020-01-01"*, limit: 10, sortowanie po polu *eventTime* malejąco
Kroki testowe	1. Uruchomienie serwera GraphQL oraz MongoDB. 2. Upewnienie się, że baza MongoDB zawiera rekordy. 3. Wykonanie zapytania GraphQL z filtrem czasowym zawsze spełnionym (data po roku 2020). 4. Analiza odpowiedzi JSON zwróconej przez API. 5. Weryfikacja obecności pól: *ID, symbol, price, quantity, eventTime*.
Oczekiwany rezultat	Zapytanie GraphQL zostaje przetworzone, filtr *eventTime.after* przekształcony na zapytanie MongoDB, wykonane zapytanie do bazy danych zwraca rekordy spełniające warunek. Odpowiedź JSON zawiera listę transakcji z wybranymi polami, posortowaną według *eventTime* malejąco.
Rzeczywisty rezultat	Zapytanie zostało wykonane poprawnie. API GraphQL zwróciło listę 10 rekordów z polami *ID, symbol, price, quantity, eventTime*. Wszystkie zwrócone rekordy spełniały warunek filtra, dane były posortowane malejąco według czasu zdarzenia.

Test potwierdził poprawne działanie resolvera GraphQL oraz integracji z warstwą repozytorium MongoDB. System poprawnie mapuje filtry GraphQL na warunki zapytań MongoDB (w tym filtry czasowe, liczbowe oraz wyrażenia regularne), wykonuje sortowanie oraz limitowanie wyników zgodnie z parametrami zapytania.

6.8 Test T8: Zapytanie GraphQL z filtrami niezwracającymi wyników

Test weryfikuje poprawność obsługi zapytań GraphQL z filtrami, które nie są spełnione przez żadne rekordy w bazie danych.

Tablica 8. Scenariusz testowy T8: Zapytanie GraphQL z filtrami niezwracającymi wyników.

Pole	**Treść**
Nazwa scenariusza	Zapytanie GraphQL z filtrem niezwracającym rekordów
Cel testu	Weryfikacja poprawności obsługi zapytań GraphQL z warunkami, które nie są spełnione przez żadne dane w bazie oraz potwierdzenie zwrócenia pustej listy bez błędów
Parametry testu	Zapytanie GraphQL typu *trades* z filtrem: *eventTime.before < "2000-01-01"*, limit: 10
Kroki testowe	1. Uruchomienie serwera GraphQL oraz MongoDB. 2. Upewnienie się, że baza MongoDB zawiera rekordy z datami nowszymi niż rok 2000. 3. Wykonanie zapytania GraphQL z filtrem czasowym nigdy niespełnionym (data przed rokiem 2000). 4. Analiza odpowiedzi JSON zwróconej przez API. 5. Weryfikacja, że odpowiedź zawiera pustą listę bez komunikatów o błędach.
Oczekiwany rezultat	Zapytanie GraphQL zostaje przetworzone poprawnie, filtr *eventTime.before* przekształcony na warunek MongoDB. Zapytanie do bazy danych nie zwraca żadnych rekordów. Odpowiedź JSON zawiera pustą listę *trades: []* bez błędów, z kodem statusu HTTP 200.
Rzeczywisty rezultat	Zapytanie zostało wykonane poprawnie. API GraphQL zwróciło pustą listę rekordów *trades: []*. Brak błędów w odpowiedzi, status HTTP 200. System poprawnie obsłużył sytuację braku wyników spełniających warunek.

Test potwierdził poprawną obsługę przypadków brzegowych przez interfejs GraphQL. System nie generuje błędów w sytuacji, gdy zapytanie z poprawnymi parametrami nie zwraca żadnych wyników, co jest zgodne z konwencjami REST i GraphQL (pusta lista vs. kod błędu 404).

6.9 Test T9: Proces ETL i mechanizm tiered storage

Test weryfikuje poprawność działania procesu ETL oraz mechanizmu przenoszenia danych z warstwy gorącej (MongoDB) do warstwy zimnej (MinIO) z jednoczesnym przetwarzaniem danych do formatu Parquet.

Tablica 9. Scenariusz testowy T9: Proces ETL i mechanizm tiered storage.

Pole	**Treść**
Nazwa scenariusza	Przenoszenie danych z MongoDB do MinIO oraz generowanie zbiorów analitycznych
Cel testu	Weryfikacja poprawności procesu ETL obejmującego: pobranie danych z MongoDB, zapis do warstwy *raw*, wzbogacenie danych w warstwie *curated*, generowanie agregacji w warstwie *analytics* oraz usunięcie danych z MongoDB po pomyślnym transferze
Parametry testu	Zbiór rekordów w MongoDB (kolekcje: *BinanceTradeEvent*, *NotificationEvent*), MinIO bucket: *data-lake*, warstwy: *raw/trades, curated/trades, analytics/minute, analytics/hourly, analytics/daily*
Kroki testowe	1. Uruchomienie MongoDB, MinIO oraz modułu ETL. 2. Upewnienie się, że MongoDB zawiera rekordy testowe. 3. Wywołanie procesu ETL. 4. Weryfikacja zapisu plików JSONL (gzip) w warstwie *raw* MinIO. 5. Weryfikacja zapisu plików Parquet w warstwie *curated* MinIO. 6. Weryfikacja zapisu plików Parquet w warstwie *analytics* (trzy poziomy: minutowy, godzinowy, dzienny). 7. Sprawdzenie kolekcji MongoDB – rekordy powinny zostać usunięte po pomyślnym transferze.
Oczekiwany rezultat	Proces ETL pobiera dane z MongoDB w partiach, zapisuje surowe dane w formacie JSONL (gzip) w folderze *raw/trades* oraz *raw/notifications*. Następnie tworzy wzbogacone rekordy *ProcessedTrade* i zapisuje je w formacie Parquet w folderze *curated*. Moduł analytics agreguje dane do poziomów czasowych i zapisuje je w folderze *analytics*. Po pomyślnym zapisie wszystkich warstw, przetworzone rekordy są usuwane z MongoDB.
Rzeczywisty rezultat	Proces ETL został wykonany pomyślnie. Pliki JSONL z surowymi danymi pojawiły się w warstwie *raw*. Pliki Parquet z przetworzonymi danymi zostały zapisane w warstwie *curated*. Pliki Parquet z agregacjami na trzech poziomach czasowych pojawiły się w folderach *analytics/minute*, *analytics/hourly*, *analytics/daily*. Kolekcje MongoDB zostały opróżnione.

Test potwierdził poprawne działanie pełnego łańcucha przetwarzania danych w modelu *tiered storage*. Proces ETL realizuje wszystkie założone etapy: ekstrakcję z warstwy gorącej, transformację danych (wzbogacanie o metadane, agregacja), zapis do warstwy zimnej w formatach zoptymalizowanych pod kątem archiwizacji oraz ładowanie danych analitycznych w strukturze partycjonowanej według symbolu i przedziałów czasowych. Mechanizm zapewnia odciążenie bazy operacyjnej MongoDB przy zachowaniu dostępności danych historycznych w magazynie obiektowym MinIO.

Rozdział 7

Podsumowanie

Celem niniejszej pracy było zaprojektowanie i implementacja kompletnego systemu do przetwarzania i archiwizacji strumieniowych danych finansowych z rynku kryptowalut w środowisku lokalnym (*on-premises*). Założenia te zostały zrealizowane w pełni — powstał działający system oparty na architekturze mikrousługowej, który integruje wszystkie zaplanowane komponenty i spełnia postawione wymagania funkcjonalne oraz niefunkcjonalne.

W ramach pracy zaimplementowano niezawodny serwer strumieniowy utrzymujący połączenia z WebSocket giełdy Binance, automatycznie wznawiający połączenie w przypadku awarii. Zbudowano kanał dystrybucyjny oparty na NATS JetStream, zapewniający buforowanie zdarzeń oraz gwarantowane dostarczenie komunikatów nawet w przypadku chwilowej niedostępności usług konsumenckich. Utworzono procesy ETL dzielące dane na zbiory surowe (*raw*) i oczyszczone (*curated*) oraz wzbogacające je o metadane analityczne. Wdrożono mechanizm pamięci warstwowej (*tiered storage*) automatycznie przenoszący dane historyczne z operacyjnej bazy MongoDB do archiwalnego magazynu obiektowego MinIO, optymalizując zasoby warstwy operacyjnej przy zachowaniu dostępności danych historycznych. Zaimplementowano interfejs GraphQL umożliwiający wykonywanie elastycznych zapytań o dane bieżące z zastosowaniem filtrów i projekcji pól. Zrealizowano bota platformy Telegram realizującego funkcję powiadamiania użytkowników o zdarzeniach rynkowych w czasie rzeczywistym.

System został pomyślnie uruchomiony i zweryfikowany pod kątem poprawności działania poszczególnych komponentów oraz przepływu danych między nimi. Testy manualne opisane w rozdziale 6 potwierdziły prawidłowe funkcjonowanie wszystkich modułów — od pobierania danych przez przetwarzanie i archiwizację, aż po udostępnianie przez API oraz wysyłanie powiadomień. Weryfikacja mechanizmu migracji danych między warstwami wykazała poprawne przenoszenie rekordów z MongoDB do MinIO bez utraty informacji.

Projekt udowodnił, że możliwe jest zbudowanie kompletnego, autonomicznego systemu przetwarzania danych rynkowych w środowisku lokalnym z wykorzystaniem otwartych technologii. Architektura mikrousługowa sterowana zdarzeniami (*Event-Driven Architecture*) okazała się odpowiednia dla

tego typu zastosowań, zapewniając luźne powiązanie między komponentami, odporność na awarie oraz możliwość niezależnego skalowania i rozwoju poszczególnych modułów.

System można rozwijać w wielu kierunkach, przede wszystkim poprzez centralizację logów z zastosowaniem identyfikatorów korelacji oraz wspólnego backendu typu Loki czy ELK, co ułatwiłoby diagnozę problemów w systemach rozproszonych. Istotnym uzupełnieniem byłaby ekspozycja metryk technicznych i domenowych do Prometheus oraz budowa dashboardów w Grafanie, umożliwiających monitorowanie opóźnień, wydajności procesów ETL oraz stanu konsumentów JetStream. Wdrożenie śledzenia rozproszonego z wykorzystaniem OpenTelemetry pozwoliłoby na wizualizację pełnej ścieżki przetwarzania zdarzeń między mikroserwisami. W warstwie aplikacyjnej warto rozważyć rozszerzenie API GraphQL o dostęp do danych historycznych z MinIO poprzez mechanizm federacji oraz wzbogacenie modułu analitycznego o bardziej zaawansowane metryki, takie jak zmienność w oknach kroczących, wykrywanie anomalii czy budowę modeli predykcyjnych w ramach potoku uczenia maszynowego. Dodatkowo, wprowadzenie alertów operacyjnych informujących o problemach działania systemu oraz skalowanie do pełnej listy symboli giełdowych wymagałoby mechanizmów fragmentacji i lepszego zarządzania backpressure. Zaprezentowana architektura stanowi solidny fundament do realizacji tych rozszerzeń.

Implementacja systemu pozwoliła na praktyczną weryfikację wybranych technologii. Język Go okazał się niezwykle efektywnym wyborem do budowy usług sieciowych i przetwarzania strumieniowego, oferując niskie zużycie zasobów przy wysokiej wydajności. Jasna składnia języka oraz wbudowana obsługa kontekstów i anulowania operacji znacząco ułatwiły implementację niezawodnych komponentów, takich jak serwer strumieniowy czy konsumenci JetStream. Baza MongoDB sprawdziła się jako warstwa operacyjna, zapewniając prostotę w modelowaniu dokumentów oraz elastyczność schematów, co jest szczególnie istotne w kontekście danych o zmiennej strukturze. W porównaniu z relacyjnymi bazami danych, MongoDB wymagała mniejszego nakładu na konfigurację i zarządzanie migracjami, co przyspieszyło iteracje rozwojowe. Istnotnym ograniczeniem okazała się sytuacja wokół MinIO — w trakcie realizacji projektu producent zaprzestał rozwijania wersji open source na rzecz w pełni komercyjnego rozwiązania enterprise. W konsekwencji wykorzystana wersja nie otrzymuje już aktualizacji, wsparcia technicznego, a oficjalna dokumentacja została usunięta, co może stanowić wyzwanie w kontekście długoterminowego utrzymania systemu. Pomimo tego baza MinIO spełniła swoją rolę w ramach demonstratora, lecz w środowisku produkcyjnym zasadne byłoby rozważenie alternatyw, takich jak SeaweedFS czy bezpośrednie wykorzystanie rozwiązań chmurowych kompatybilnych z API S3.

Podsumowując, wszystkie cele szczegółowe określone we wstępie zostały osiągnięte. System działa zgodnie z założeniami, spełnia wymagania funkcjonalne i niefunkcjonalne, a przeprowadzone testy potwierdzają jego poprawność i stabilność.

Bibliografia

[1] *Total Crypto Market Cap Chart*, dostęp uzyskano 2026-01-18. adr.: https://www.coingecko.com/en/charts.

[2] Cong, L. W., Li, X., Tang, K. i Yang, Y., "Crypto Wash Trading," *SSRN Electronic Journal*, 2023. DOI: 10.2139/ssrn.3530220.

[3] Makarov, I. i Schoar, A., "Trading and Arbitrage in Cryptocurrency Markets," *Journal of Financial Economics*, 2018. DOI: 10.2139/ssrn.3171204.

[4] Satyanarayanan, M., "The Emergence of Edge Computing," 2017. DOI: 10.1109/MC.2017.9.

[5] Jones, T. i in., "UNITY: Unified Memory and File Space," czer. 2017, s. 1–8.

[6] Golab, L. i Özsu, M. T., "Issues in data stream management," *ACM SIGMOD Record*, t. 32, nr. 2, s. 5–14, 2003.

[7] Vlachos, M., Wu, K.-L., Chen, S.-K. i Yu, P., "Correlating burst events on streaming stock market data," *Data Min. Knowl. Discov.*, t. 16, s. 109–133, lut. 2008. DOI: 10.1007/s10618-007-0066-x.

[8] Hackathorn, R., "The BI watch real-time to real-value," *DM Review*, t. 14, lut. 2004.

[9] Stonebraker, M., Cetintemel, U. i Zdonik, S., "The 8 requirements of real-time stream processing," *SIGMOD Record*, t. 34, s. 42–47, grud. 2005. DOI: 10.1145/1107499.1107504.

[10] Marz, N. i Warren, J., *Big Data: Principles and best practices of scalable realtime data systems*. Manning Publications Co., 2015.

[11] Kreps, J., *Questioning the Lambda Architecture*, O'Reilly Radar, dostęp uzyskano 2026-01-21, 2014. adr.: https://www.oreilly.com/radar/questioning-the-lambda-architecture/.

[12] Kleppmann, M., *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media, 2017, Rozdział: Message Brokers, ISBN: 978-1449373320.

[13] Akidau, T. i in., "The dataflow model: A practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing," *Proceedings of the VLDB Endowment*, t. 8, sierp. 2015. DOI: 10.14778/2824032.2824076.

[14] Carbone, P., Katsifodimos, A., Ewen, S., Markl, V., Haridi, S. i Tzoumas, K., "Apache Flink: Stream and Batch Processing in a Single Engine," *IEEE Data Engineering Bulletin*, t. 36, s. 28–38, 2015.

[15] Beyer, B., Jones, C., Petoff, J. i Murphy, N. R., *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, Inc., 2016, ISBN: 9781491929124.

[16] Gedik, B., Schneider, S., Hirzel, M. i Wu, K.-L., "Elastic Scaling for Data Stream Processing," *Parallel and Distributed Systems, IEEE Transactions on*, t. 25, s. 1447–1463, czer. 2014. DOI: 10.1109/TPDS.2013.295.

[17] Eugster, P., Felber, P., Guerraoui, R. i Kermarrec, A.-M., "The Many Faces of Publish/Subscribe," *ACM Comput. Surv.*, t. 35, s. 114–131, czer. 2003. DOI: 10.1145/857076.857078.

[18] Hohpe, G. i Woolf, B., *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley Professional, 2004, ISBN: 978-0321200686.

[19] Richards, M. i Ford, N., *Fundamentals of Software Architecture: An Engineering Approach*. O'Reilly Media, 2020, ISBN: 978-1492043454.

[20] Brogaard, J., Hagströmer, B., Nordén, L. i Riordan, R., "Trading Fast and Slow: Colocation and Liquidity," *Review of Financial Studies*, t. 28, s. 3407–3443, grud. 2015. DOI: 10.1093/rfs/hhv045.

[21] CoinGecko Research, *2025 Cryptocurrency Exchange Market Report*, Raport roczny, sekcja Market Share, dostęp uzyskano 2026-01-18, 2025. adr.: https://www.coingecko.com/research/publications/centralized-crypto-exchanges-market-share.

[22] Łasocha, W. i Badurowicz, M., "Comparison of WebSocket and HTTP protocol performance," *Journal of Computer Sciences Institute*, t. 19, s. 67–74, czer. 2021. DOI: 10.35784/jcsi.2452.

[23] Fielding, R. i Reschke, J., "Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content," Internet Engineering Task Force, RFC 7231, czer. 2014, Sekcja 4.2.2 Idempotent Methods, dostęp uzyskano 2026-01-18. adr.: https://www.rfc-editor.org/rfc/rfc7231#section-4.2.2.

[24] Margański, P. i Pańczyk, B., "REST and GraphQL comparative analysis," *Journal of Computer Sciences Institute*, t. 19, s. 89–94, czer. 2021. DOI: 10.35784/jcsi.2473.

[25] Cherkashyn, V., *Binance-Data-Collector-and-Visualization*, Repozytorium GitHub, dostęp uzyskano 2026-01-02. adr.: https://github.com/VladyslavCherkashyn/Binance-Data-Collector-and-Visualization.

[26] Gonzalez, A., *binance_ticker_rust*, Repozytorium GitHub, dostęp uzyskano 2026-01-02. adr.: https://github.com/arturogonzalezm/binance_ticker_rust.

[27] lostleaf, *binance_datatool (BHDS – Binance Historical Data Service)*, Repozytorium GitHub, dostęp uzyskano 2026-01-02. adr.: https://github.com/lostleaf/binance_datatool.

[28] Schneider, P. J., *binance-websocket*, Repozytorium GitHub, dostęp uzyskano 2026-01-02. adr.: https://github.com/pjschneiCMU/binance-websocket.

[29] Pike, R., "Go at Google," dostęp uzyskano 2026-01-18, paź. 2012, s. 5–6. DOI: 10.1145/2384716.2384720. adr.: https://go.dev/talks/2012/splash.article.

[30] *NATS Docs*, dostęp uzyskano 2026-01-18, 2024. adr.: https://docs.nats.io/nats-concepts/jetstream.

[31] Davoudian, A., Chen, L. i Liu, M., "A survey on NoSQL stores," *ACM Computing Surveys*, t. 51, s. 1–43, kw. 2018. DOI: 10.1145/3158661.

[32] Gadban, F. i Kunkel, J., "Analyzing the Performance of the S3 Object Storage API for HPC Workloads," *Applied Sciences*, 2021. DOI: https://doi.org/10.3390/app11188540.

[33] Thusoo, A. i in., "Hive - A Petabyte Scale Data Warehouse Using Hadoop," *Proceedings - International Conference on Data Engineering*, s. 996–1005, sty. 2010. DOI: 10.1109/ICDE.2010.5447738.

[34] Merkel, D., "Docker: lightweight Linux containers for consistent development and deployment," *Linux Journal*, t. 2014, mar. 2014.

[35] MongoDB, Inc., *MongoDB Manual*, dostęp uzyskano 2026-01-18, 2024. adr.: https://www.mongodb.com/docs/manual/.

[36] Korada, L. i Sikha, V., "Why are large enterprises building private clouds after their journey on public clouds?," t. 11, s. 49–52, lut. 2024. DOI: 10.5281/zenodo.13326719.

[37] MinIO, Inc., *MinIO Object Storage Documentation*, dostęp uzyskano 2026-01-18, 2024. adr.: https://github.com/minio/docs.

[38] GraphQL Foundation, "GraphQL," spraw. tech., wrz. 2025, dostęp uzyskano 2026-01-18. adr.: https://spec.graphql.org/September2025/.

[39] 99designs, *gqlgen: Type-safe GraphQL for Go*, dostęp uzyskano 2026-01-18. adr.: https://gqlgen.com/.

[40] The Apache Software Foundation, *Apache Parquet Documentation*, dostęp uzyskano 2026-01-18. adr.: https://parquet.apache.org/docs/overview/.

[41] Francia, S., *Viper: Go Configuration with fangs*, Repozytorium GitHub, dostęp uzyskano 2026-01-02. adr.: https://github.com/spf13/viper.

[42] Project Jupyter, *Jupyter Notebook Documentation*, dostęp uzyskano 2026-01-18. adr.: https://docs.jupyter.org/en/latest/.

[43] Newman, S., *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media, 2015, ISBN: 978-1491950357.

[44] Faseeha, U., Syed, H., Samad, F., Zehra, S. i Ahmed, H., "Observability in Microservices: An In-Depth Exploration of Frameworks, Challenges, and Deployment Paradigms," *IEEE Access*, t. PP, sty. 2025. DOI: 10.1109/ACCESS.2025.3562125.

[45] Taherizadeh, S. i Stankovski, V., "Dynamic Multi-level Auto-scaling Rules for Containerized Applications," *The Computer Journal*, t. 62, maj 2018. DOI: 10.1093/comjnl/bxy043.

[46] Apollo Graph Inc., *Apollo Docs: Introduction to Apollo Federation*, dostęp uzyskano 2026-01-18, 2024. adr.: https://www.apollographql.com/docs/graphos/schema-design/federated-schemas/federation.

Załączniki

Kod źródłowy systemu jest dostępny w publicznym repozytorium GitHub pod adresem:

https://github.com/amdla/BTCStreamAnalysis/